



Escuela
Politécnica
Superior

Tutor Virtual Inteligente para Aprendizaje Personalizado



Grado en Ingeniería Informática

Trabajo Fin de Grado

Autor:
Álvaro Cerezo Cabañero
Tutor/es:
Félix Escalona Moncholí
Alberto Joaquín López Sellers

Julio 2026



Universitat d'Alacant
Universidad de Alicante

Tutor Virtual Inteligente para Aprendizaje Personalizado

Autor

Álvaro Cerezo Cabañero

Tutor/es

Félix Escalona Moncholí

B161 Ciencia de la computación e int. artif.

Alberto Joaquín López Sellers

B161 Ciencia de la computación e int. artif.



Grado en Ingeniería Informática



Escuela
Politécnica
Superior



Universitat d'Alacant
Universidad de Alicante

ALICANTE, Julio 2026

Preámbulo

Este trabajo busca estudiar el uso de tutores inteligentes en un marco universitario, aplicado al grado de ingeniería informática, valiéndose de un LLM (Large Language Model) con RAG. Los avances en la IA ocurridos en los últimos años han abierto nuevas posibilidades que facilitan el día a día de las personas. En concreto, los tutores inteligentes no son una novedad reciente, pero gracias a los avances y popularización de estos modelos, han evolucionado en gran medida, de forma que el tutor pueda responder con un alto grado de coherencia y lógica en un formato conversacional. Los tutores inteligentes virtuales brindan conocimiento personalizado al alumno, adaptándose a su nivel y sirviendo como una herramienta útil que apoye el aprendizaje del alumno. El tutor inteligente creado en este proyecto, puede responder a dudas del alumno, resumir contenido de los temarios estudiados, o generar preguntas para evaluar al alumno, pudiendo adaptar así su nivel de complejidad de forma dinámica.

Agradecimientos

Este trabajo no habría sido posible sin la ayuda de mis tutores, Felix Escalona Moncholí y Alberto Joaquín López Sellers. Su ayuda, para resolver mis dudas, guiarme en el desarrollo del trabajo y su disponibilidad para responder estas cuestiones de forma rápida, han sido fundamentales para poder realizarlo.

También agradezco a mi familia, mis padres y mi hermano, por el apoyo y los ánimos recibidos, sin ellos me sería imposible haber cursado el grado de ingeniería informática.

Por último, también agradezco el apoyo recibido, a lo largo del grado y del desarrollo del TFG, a los compañeros de mi curso, con los cuales he podido entablar una buena relación y nos hemos ayudado tanto de forma académica como personal. En especial a Alejandro Romero López y Aitor López Romero, que me han acompañado a lo largo de estos años de carrera universitaria e incluso otros muchos antes de empezarla.

*Dedicado a mi familia y amigos,
muchas gracias por vuestro apoyo.*

Índice general

1	Introducción	1
2	Marco Teórico	3
3	Estado del arte	5
3.1	Antecedentes históricos	5
3.2	Primeros ITS reales	6
3.2.1	SCHOLAR	6
3.2.2	Autotutor	6
3.3	Inicios del siglo XXI	7
3.3.1	Geometry cognitive Tutor	8
3.4	Actualidad	8
3.4.1	LPITutor	9
3.4.2	ITAS	9
4	Objetivos	11
5	Metodología	13
5.1	Tecnologías	13
5.1.1	Entorno	13
5.1.2	Librerías	15
5.2	Ingeniería de prompts	16
5.2.1	Estructura y componentes de un prompt	16
5.2.2	Principios fundamentales de diseño	17
5.2.3	Técnicas y estrategias de interacción	18
6	Desarrollo	21
6.1	Funcionamiento básico del sistema	22
6.1.1	Router	23
6.1.2	Seleccionar tema	23
6.1.3	Formato en que el modelo devuelve las respuestas	24
6.2	Responder dudas del usuario	25
6.3	Resumir contenidos	26
6.4	Realizar preguntas	26
6.4.1	Obtención de contexto para generar la pregunta	27
6.4.2	Conocimiento y niveles de Bloom	27
6.4.3	Evaluación de respuestas	31
6.4.4	Actualizar el nivel de conocimiento del usuario con IRT	31
6.4.5	Test CBM	33

6.4.6	Generar preguntas mediante IA y Presets de preguntas	34
6.5	Recordar temas	35
6.6	Interfaz gráfica	35
7	Resultados	39
7.1	Métricas medidas	39
7.1.1	Latencia	39
7.1.2	BERTScore	40
7.1.2.1	Primera prueba de BERTscore	41
7.1.2.2	Segunda prueba con BERTscore	42
7.1.2.3	Tercera prueba con BERTscore	43
7.1.3	LLMs actuando como jueces	46
7.2	Encuestas a usuarios	51
8	Conclusiones	55
8.1	Problemas enfrentados	55
8.2	Posibles mejoras en futuras versiones	55
9	Anexo	57
9.1	Prompt usado en LLM As Judges	57
9.2	Ejemplos de conversaciones	58
	Bibliografía	65

Índice de figuras

2.1	Arquitectura de un tutor virtual inteligente basado en modelos de lenguaje . . .	4
3.1	Maquina de enseñanza de pressey, Teaching machine. Smithsonian (s.f.) . . .	5
3.2	Interfaz de autotutor	7
5.1	Diagrama de la base de datos	14
5.2	Proceso de generación de preguntas usando AutoCot	18
6.1	Diagrama principal del flujo de la aplicación	22
6.2	Diagrama de pasos que sigue un mensaje para generar una respuesta	24
6.3	Esquema de funcionamiento de RAG	26
6.4	Función IRT 2PL para el cálculo de probabilidad de acierto	31
6.5	Gráfica de $P(\theta)$ en IRT 1PL	32
6.6	Actualización del conocimiento del alumno mediante ERS-IRT (Pelánek, 2016)	32
6.7	Función IRT 3PL para el cálculo de probabilidad de acierto	33
6.8	Diagrama del funcionamiento del sistema de preguntas	34
6.9	Repasar temarios, visto desde la consola	35
6.10	Pantalla de login de la aplicación con Chainlit	36
6.11	Pantalla principal de mensajes con Chainlit	36
6.12	Interfaz de consola de la aplicación	37
7.1	Funciones de las métricas de BERTScore : Recall (R_{BERT}), Precision (P_{BERT}) y F1 (F_{BERT})	40
7.2	Resultados de la evaluación con el método "LLM as judges" para la primera pregunta	47
7.3	Resultados de la evaluación con el método "LLM as judges" para la segunda pregunta	49
7.4	Resultados de la evaluación con el método "LLM as judges" para la tercera pregunta	50
7.5	Resultados de las encuestas a usuarios (1).	52
7.6	Resultados de las encuestas a usuarios (2).	53

Índice de cuadros

6.1	Tabla de niveles taxonómicos de Bloom	28
6.2	Umbral según conocimiento para avanzar al siguiente nivel de Bloom en el código de la aplicación	29
7.1	Medición de la latencia para diferentes operaciones en el sistema	39
7.2	Resultados de la primera prueba con BERTscore	42
7.3	Resultados de la segunda prueba con BERTscore	43
7.4	Resultados de la tercera prueba con BERTscore	45
7.5	Preguntas a evaluar con "LLM as judges"	46

1 Introducción

La educación es un pilar básico de cualquier sociedad, que impulsa el desarrollo económico, cultural y social de las mismas. Su importancia ha sido reconocida a nivel global desde la Declaración Universal de los Derechos Humanos de 1948, en la que se proclamó el derecho de toda persona a la educación y se estableció la obligatoriedad y gratuidad de la educación primaria. Debido a su importancia ha ido evolucionando desde la antigüedad, donde solo unos pocos privilegiados tenían acceso a esta, hasta la actualidad, donde el sistema educativo es accesible para toda la sociedad en la mayoría de países. En las últimas décadas, los avances tecnológicos han jugado un papel clave dentro de la evolución de la educación.

La aplicación de la tecnología a los procesos educativos ha conllevado grandes cambios en la forma de aprender, han surgido plataformas para la gestión del aprendizaje, acceso a materiales docentes, aplicaciones o juegos educativos y diversas formas de fomentar el aprendizaje. Sin embargo estos ejemplos no resuelven uno de los principales problemas de la educación, la imposibilidad de ofrecer educación individual y personalizada para cada estudiante. En entornos masificados, donde un único docente debe atender a decenas de alumnos simultáneamente, resulta prácticamente imposible adaptar el ritmo, el contenido y la metodología a las necesidades individuales de cada persona. Los alumnos pueden ver reducido su rendimiento y empeorada la calidad de su enseñanza, algunos de ellos pueden incluso quedar atrasados al resto de la clase. Los tutores inteligentes virtuales buscan solucionar este problema, son un apoyo a la educación, capaces de brindar una enseñanza personalizada con un enfoque conversacional y de resolver las dudas que plantee el alumno. Investigaciones clásicas como la realizada por Bloom (1984) han demostrado que la tutoría individualizada puede mejorar significativamente el rendimiento académico frente a la enseñanza en grupo, con mejores resultados en más de un noventa por ciento de los casos (dos desviaciones estándar por encima del promedio en los resultados). En base a esto, cualquier sistema que busque ofrecer tutorías personalizadas resulta práctico e interesante para mejorar la calidad de la enseñanza.

Huang y cols. (2025) demuestran la efectividad de los tutores inteligentes con su meta análisis en base a otros trabajos anteriores sobre la materia. Los resultados de su investigación muestran mejoras significativas en la actitud de aprendizaje y las puntuaciones de los exámenes. Sin embargo, inciden en que otros aspectos como la resolución de problemas no tienen mejoras significativas, por lo que también deben tenerse en cuenta otros factores como el diseño del curso académico y el entorno educativo. Los tutores inteligentes han demostrado ser una ayuda útil para los estudiantes.

Para este trabajo, se ha desarrollado un tutor universitario centrado en las asignaturas de ingeniería informática para probar su capacidad tanto con contenidos teóricos como a la hora de trabajar con código. El modelo puede generar respuestas precisas y contextualizadas

gracias a que accede a los materiales docentes de las asignaturas. En una fase inicial del proyecto, también se consideró la posibilidad de adoptar un enfoque orientado a la enseñanza primaria, más generalista y sin la necesidad de recopilar datos específicos. Sin embargo, este planteamiento implicaba desafíos adicionales que se alejaban del objetivo principal del trabajo, tales como la adaptación del sistema a usuarios menores de edad o el refuerzo de medidas de seguridad necesarias en ese contexto.

2 Marco Teórico

Los tutores inteligentes virtuales o por sus siglas en inglés ITS (*Intelligent Tutor System*, Cataldi y Lage (2009)), consisten en programas informáticos que emulan ser un tutor humano real, entablan una conversación con el alumno y deciden qué enseñar y cómo hacerlo, adaptándose al nivel del alumno con diferentes estrategias pedagógicas. Hoy en día han mejorado mucho gracias a la aparición de los LLM (Large Lenguaje Models) en los últimos años. Los primeros tutores consistían en sistemas basados en reglas y eran más simples. En la actualidad son capaces de mantener conversaciones largas y coherentes con el alumno, con la capacidad de dar respuestas mucho más detalladas y personalizadas.

Los LLM en los que se basan los ITS modernos, son ampliamente conocidos en la actualidad, ejemplos de estos pueden ser los famosos ChatGPT, Gemini o Claude; modelos de inteligencia artificial que procesan el lenguaje natural y generan respuestas basadas en probabilidad. Para funcionar se basan en Deep Learning gracias a redes neuronales.

Las respuestas que da el ITS dependen de la IA y de los materiales que consulte para generar su respuesta, por lo que no se puede obviar que no tiene un entendimiento semántico real de lo que dice, si no que da respuestas basadas en patrones y probabilidad, que no están libres de alucinaciones. En este sentido, un tutor inteligente virtual no va a remplazar a un tutor humano, aunque también presente ventajas, como la disponibilidad o la escalabilidad.

Podemos dividir su arquitectura, a nivel de diseño, en 3 partes o módulos representados en la figura 2.1.

- **Módulo experto:** Contiene el conocimiento del dominio que se debe impartir al alumno. Contiene la información en bases de datos y usa redes neuronales o sistemas basados en reglas, para extraer dicho conocimiento y generar respuestas precisas con las que responder al alumno.
- **Módulo del estudiante:** Módulo dinámico, basado en las interacciones del programa con el alumno. Representa el conocimiento o nivel que el ITS cree que tiene el estudiante, y se actualiza de forma continua a partir de sus respuestas. Permite que se pueda enseñar al alumno de forma personalizada.
- **Módulo tutorial:** Toma las decisiones pedagógicas, como determinar qué estrategia seguir para responder a las cuestiones planteadas por el alumno. También, aprovechando el módulo del estudiante, decide el nivel de dificultad con el que debe plantear las actividades y conocimiento a impartir.

Aparte de estos 3 módulos principales, también está la interfaz con la que interactúa el usuario.

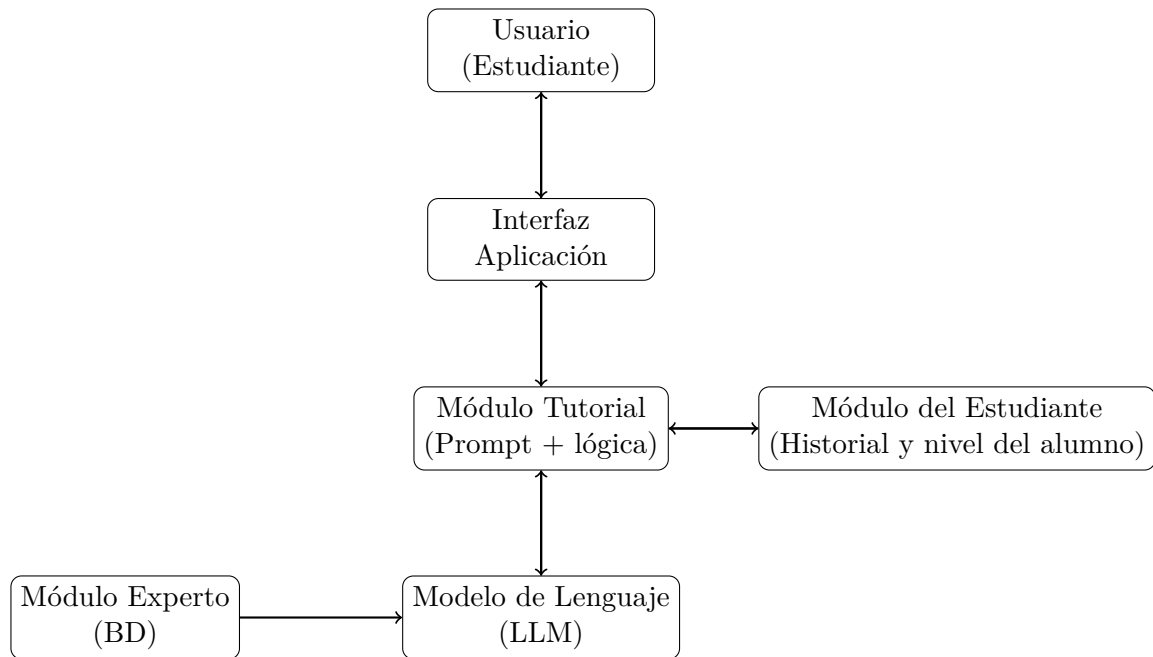


Figura 2.1: Arquitectura de un tutor virtual inteligente basado en modelos de lenguaje

3 Estado del arte

La idea de un tutor inteligente que enseñe a un alumno sin la necesidad de un profesor no es un concepto novedoso. A continuación se citan algunos de los modelos más relevantes, junto a los algoritmos principales que los hacen funcionar. A la par, se realiza un recorrido histórico, desde los primeros modelos mecánicos hasta la actualidad.

3.1 Antecedentes históricos

El primer antecedente histórico surge en 1924, cuando Sidney Pressey, profesor de psicología en la universidad de Ohio, inventó una máquina que realizaba preguntas al alumno y mostraba cuatro posibles respuestas, de las que el alumno escogía una. La máquina además daba una puntuación a los alumnos según las preguntas acertadas. Además, podía configurarse para que sólo avanzase cuando el alumno marcaba la respuesta correcta. Con los estándares actuales, no se puede considerar inteligente a esta máquina, sólo muestra las preguntas y calcula la puntuación del alumno. De hecho, aunque el objetivo original de Pressey fuese una máquina auto-calificadora para los alumnos, con el paso del tiempo demostró su capacidad para enseñar y generar conocimiento con su uso.

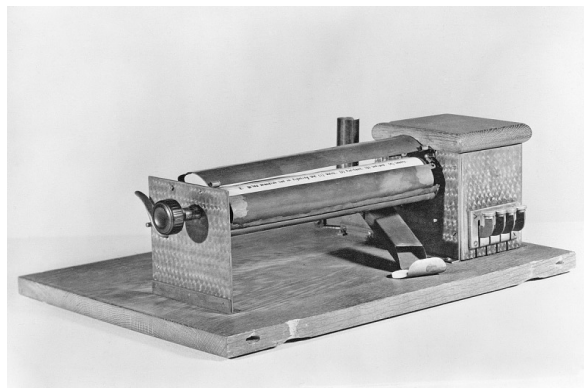


Figura 3.1: Máquina de enseñanza de pressey, Teaching machine. Smithsonian (s.f.)

El psicólogo Burrhus Frederic Skinner, creó otra máquina, similar a la de Pressey, durante la década de 1950, aplicando sus ideas pedagógicas. Skinner creía que el aprendizaje debía ser dirigido por refuerzos positivos, con retroalimentación positiva instantánea, el ritmo se podía ajustar según la persona, avanzando de forma gradual según acertase preguntas.

3.2 Primeros ITS reales

El desarrollo de los primeros tutores inteligentes propiamente dichos comienza en la década de 1970. Gracias a la aplicación de los primeros sistemas de inteligencia artificial, mucho más rudimentarios que los actuales, a la educación.

3.2.1 SCHOLAR

Uno de los primeros ejemplos relevantes es *SCHOLAR*, Carbonell y Collins (1970). Este sistema estaba diseñado para enseñar geografía de América del Sur y es considerado uno de los primeros tutores inteligentes capaces de interactuar con el estudiante utilizando lenguaje natural. *SCHOLAR* utilizaba una representación semántica del conocimiento basada en redes (Base de datos estructurada con relaciones entre los conceptos). El sistema convertía el texto a estructuras lógicas que buscaba en la red semántica. Esto permitía generar de forma dinámica preguntas nuevas o responder a preguntas del usuario, manteniendo un diálogo básico con el usuario.

Gracias a *SCHOLAR* y otros ITS que surgieron en las siguientes décadas, se define la arquitectura clásica de los ITS: Módulo tutorial, módulo experto y módulo del estudiante.

3.2.2 Autotutor

A finales de los 90, surge otro de los ITS más relevantes, Autotutor, D'Mello y Graesser (2012). A diferencia de otros tutores virtuales, se enfoca en mantener un diálogo que resulte natural con conversaciones continuas que pueden ser por texto o voz, esforzándose en emular a un tutor humano. Sus creadores defendían que un tutor particular para cada alumno era más efectivo que las clases tradicionales, pero imposible de realizar por los costes monetarios, lo que provocaba que algunos alumnos se quedasen atrasados. Por ello se centraron en simular los aspectos pedagógicos y emocionales de un tutor humano aplicados a un sistema informático que resultase más económico.

Para generar las respuestas usa análisis del lenguaje natural (*NLP*), tiene en cuenta los conocimientos previos del alumno e inclusive detecta con sensores las emociones del usuario, mediante el análisis de los gestos faciales, lenguaje corporal y forma de hablar. Pudiendo cambiar su estrategia según estos. Dispone de varias estrategias de aprendizaje cognitivo, presenta preguntas abiertas desafiantes que requieren de razonamientos y explicaciones, en las que colabora con el alumno hasta construir la respuesta completa. En el módulo experto tiene, además de los contenidos curriculares a enseñar, errores comunes y respuestas ideales, que emplea para dar retroalimentación, y así expandir ideas incompletas y corregir malentendidos, dentro de su enfoque en conversaciones humanas con distintas estrategias pedagógicas según la situación. Además, tal y como se ve en la figura 3.2, dispone de un modelo animado para representar al tutor, de nuevo, con el mismo objetivo de emular una persona real. También dispone del historial de mensajes previos. Dentro de este podemos observar un ejemplo de como ayuda a expandir el conocimiento participando en construir la respuesta.

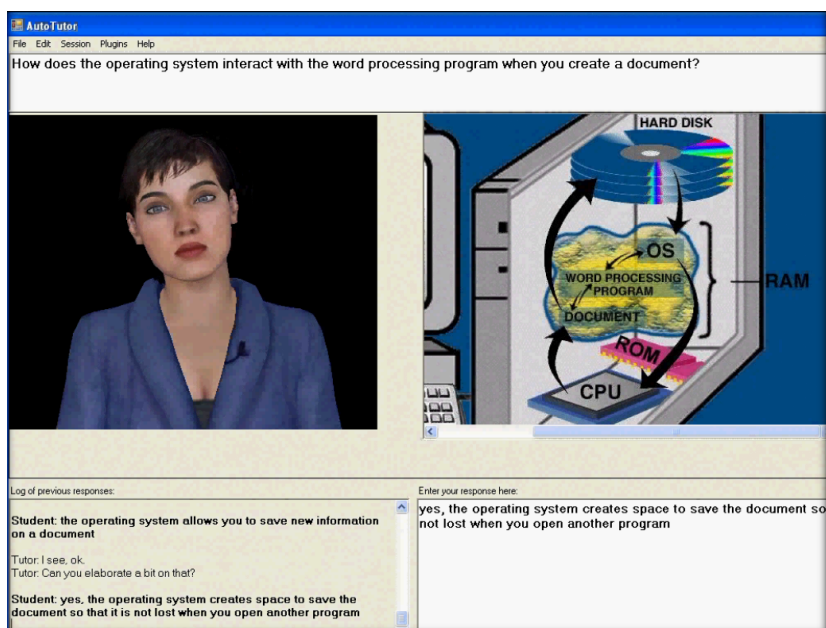


Figura 3.2: Interfaz de autotutor

Su análisis del lenguaje natural (*NLP*) consiste en varios algoritmos trabajando juntos, de los que el principal y más importante es el análisis semántico latente (*Latent semantic analysis*, LSA Venegas (2003)). LSA analiza las relaciones entre conjuntos de documentos (En este caso, solo dos, respuesta recibida y la respuesta esperada), asume que las palabras con un significado cercano aparecerán en fragmentos de texto similares en los conjuntos (hipótesis de la distribución). En base a esto construye una matriz con un recuento de palabras que aparecen en cada conjunto. Además, reduce el número de filas mediante otra técnica, descomposición de valores singulares (*SVD*) mientras preserva la estructura. Después compara dos columnas de la matriz (conjuntos, que son la respuesta recibida y la esperada) por similitud de cosenos, obteniendo un número en el rango del menos uno a uno, cuanto más cercano a uno sea el resultado, más parecidos son los conjuntos.

Los ITS surgidos durante estos años, como Autotutor suponen un avance enorme tecnológico respecto a SCHOLAR y los ITS primigenios surgidos en las anteriores décadas, sin embargo, las IAs de estos sistemas dependían de cientos de reglas definidas que moldeaban su comportamiento, lo que los convertía en sistemas costosos y nada escalables.

3.3 Inicios del siglo XXI

A partir del 2000 comienzan a idearse sistemas basados en probabilidad, en lugar de los sistemas deterministas basados en reglas que se programaban en anteriores décadas. En estos modelos el sistema estima por probabilidad lo que el usuario sabe, en lugar de dar por sentado que sabe algún dato en concreto. A pesar de esto, en esta época es común que los modelos sigan basándose en reglas y árboles de decisiones, por lo que surgen sistemas híbridos.

En la década de el 2010 comienzan a ganar relevancia el deep learning, por lo que los sistemas publicados en estos años son más flexibles y potentes que los anteriores, porque

utilizan modelos de IA predecesores a los LLMs.

3.3.1 Geometry cognitive Tutor

Fue un ITS desarrollado en la Universidad Mellon Carnegie, en los Estados Unidos, que surgió en los primeros años del 2000. Perteneció a la familia de tutores cognitivos (CTAT Guillen y cols. (2017)). Utiliza un modelo cognitivo para dar retroalimentación en tiempo real a los usuarios, gracias a registrar los pasos que siguen. El modelo aprende mediante el aprendizaje activo e intenta reproducir como un humano percibe y piensa ante los problemas de distintas maneras. Es capaz de reproducir el conocimiento de un experto o ajustarse al nivel de un alumno, simulando el razonamiento que sigue y generar así múltiples soluciones a los problemas que presenta a los estudiantes. Para poder simular el conocimiento del estudiante, realiza estimaciones probabilísticas basadas en rastreo de conocimiento bayesiano (*Bayesian Knowledge Tracing* ,BKT Bulut y cols. (2023)). El algoritmo BKT asume que el conocimiento del alumno se representa con un conjunto de variables binarias (1 o 0). Cada variable representa una habilidad, o parte del conocimiento sobre la materia, y BKT determina si las domina o no las domina. Estas variables se actualizan con las interacciones con el usuario y son las que determinan como responde el tutor. Para actualizar el valor realiza cálculos sobre diversas probabilidades:

- Probabilidad de que alumno conozca una nueva habilidad de antemano.
- Probabilidad de que el usuario falle al aplicar una habilidad que conoce.
- Probabilidad de que aplique correctamente una habilidad no aprendida
- Probabilidad de demostrar conocimiento sobre una habilidad nueva recién aprendida

Gracias a BKT, puede dar respuestas personalizadas y mostrar preguntas acordes al nivel del alumno.

3.4 Actualidad

En la actualidad, el desarrollo de modelos de lenguaje de gran tamaño (*Large Language Models*, LLMs) ha supuesto un avance significativo para los tutores inteligentes. Estos modelos permiten generar respuestas coherentes y contextualizadas en lenguaje natural, facilitando una interacción más fluida con el estudiante. A diferencia de los sistemas tradicionales, no están limitados por reglas predefinidas y pueden generalizar el conocimiento y adaptarse a distintos dominios, por lo que son mucho más personalizables y flexibles. No obstante, estos sistemas también presentan desafíos, como la posibilidad de generar información incorrecta.

Gracias a la popularización de los LLM, existen muchas aplicaciones, disponibles en cualquier plataforma, basadas en IA y con carácter educativo. Sin embargo, no todas son ITS, pues no cuentan con un módulo de conocimiento del estudiante ni con estrategias de enseñanza concretas. Un ejemplo de esto sería la aplicación "Gauth", muy popular gracias a su capacidad de escanear ejercicios y resolverlos mediante IA, pero no es un ITS. Khanmigo, otra aplicación educativa, sí que contaría como ITS, personaliza el aprendizaje al nivel del alumno con un estilo socrático, priorizando el razonamiento del alumno por encima de encontrar una solución directa.

3.4.1 LPITutor

Sistema basado en LLM que aprovecha la generación aumentada por recuperación (En inglés: *Augmented retrieval generation*, RAG Liu y cols. (2025)) e ingeniería de prompts para generar respuestas personalizadas con adaptación al nivel del alumno. La ingeniería de prompts emplea una estrategia de dos capas que aplica al LLM. Una estática con plantillas pedagógicas y una segunda dinámica que incorpora contenido y metadatos recuperados específicos del alumno a la estructura del mensaje. Permitiendo ajustar el estilo, profundidad y complejidad de la respuesta al alumno. Además, usa RAG, sistema que permite recuperar información de fuentes externas e incorporarla a la solicitud del usuario, formando un prompt más complejo que enviar al LLM. El sistema RAG es de especial interés para el proyecto y se explica durante el apartado de desarrollo.

3.4.2 ITAS

ITAS es uno de los sistemas más modernos de ITS, probablemente el más reciente, el departamento de ciencias de la computación de la universidad de Cornell, en Nueva York, publicó el ensayo el 27 de abril de este mismo año, Elhaimeur y Chrisochoides (2026). El sistema se basa en el uso de múltiples agentes (LLMs) para enseñar computación cuántica. Al recibir un mensaje de un usuario, tres agentes especializados en campos concretos de la computación cuántica trabajan en paralelo, coordinados por un cuarto agente sintetizador que combina sus respuestas en una sola. La especialización reduce la alucinación pues ningún agente razona fuera del ámbito para el que fue diseñado. Cuenta con su infraestructura en la nube para reducir costes y hacerlo escalable y seguro para desplegarlo en entornos educativos reales. Han conseguido que los costes monetarios de ITAS por alumno sean menores a un libro de texto tradicional. Sin embargo, ITAS no ha sido probado lo suficiente para demostrar su utilidad en una clase. Sus desarrolladores siguen trabajando en mejorarlo actualmente, por ejemplo, buscan incluir la posibilidad de generar preguntas para los alumnos.

4 Objetivos

El objetivo es desarrollar un tutor inteligente virtual basado en LLM, que permita proporcionar apoyo educativo personalizado mediante interacción en lenguaje natural. A su vez, como objetivos intermedios para alcanzar este se plantean:

- Analizar el estado del arte de los sistemas de tutoría inteligente y su evolución hacia modelos actuales basados en LLM.
- Implementar un prototipo funcional que permita la interacción con el usuario mediante lenguaje natural.
- Integrar mecanismos de recuperación de información (RAG) para mejorar la precisión de las respuestas.
- Evaluar el conocimiento de los usuarios de forma dinámica mediante IRT.

5 Metodología

En este apartado se describen cómo se ha trabajado y las tecnologías empleadas en el desarrollo del prototipo del tutor inteligente.

5.1 Tecnologías

5.1.1 Entorno

- **Conda:** Entorno de python para gestionar y descargar con facilidad paquetes. Concretamente se ha usado miniconda, versión reducida y ligera de Conda. Cada entorno que se cree puede tener distintas versiones de python y distintos paquetes. Por ejemplo, podríamos tener el LLM y la aplicación en entornos distintos con distintas versiones de python, según la necesidad del sistema. Para el proyecto se utiliza Python 3.13.12
- **VLLM:** *VLLM Cli guide* (s.f.). Se trata de un framework sencillo y flexible capaz de servir con facilidad LLMs. Recibe un modelo y lo carga en local, gracias a la GPU, por tanto depende de la VRAM del computador. Para funcionar correctamente, necesita una gráfica Nvidia con CUDA y sistema operativo Linux.

VLLM tiene integración directa con Hugging Face, página web que es un hub y repositorio de LLMs. En este caso, obtiene y ejecuta Qwen3-0.6B.

Para el prototipo se han empleado dos LLMs: Qwen/Qwen2.5-14B-Instruct-GPTQ-Int8 y Qwen3-0.6B: El primero es una versión de mayor capacidad con 14000 millones de parámetros y preparado para seguir instrucciones y conversar (indicado por el Instruct). Crea respuestas lógicas y de calidad sin embargo requiere muchos más recursos que el otro modelo, se ha usado para realizar pruebas y ver la capacidad del prototipo solamente, pues se debía cargar en un servidor externo. Durante el desarrollo se ha usado Qwen3-0.6B, mucho más ligero y básico, con 600 millones de parámetros. Es un modelo ideal para sistemas con recursos limitados, por tanto se ha usado en local durante el desarrollo para no depender de la disponibilidad del servidor, aunque sus respuestas son demasiado simples como para ser utilizadas en una aplicación profesional.

Al cargar un modelo con VLLM se deben de tener en cuenta los requisitos del modelo y limitaciones del sistema para intentar sacar el mayor provecho al LLM dentro de las capacidades de donde se esté ejecutando, para ello, al iniciar el llm se pueden usar parámetros en el comando, como `gpu-memory-utilization` que indica cuanta memoria de la GPU se debe usar, o `max_model_len` para indicar la longitud máxima de la secuencia (prompt y output) que puede procesar el modelo.

- **SQLITE:** Sistema de bases de datos relacional, ligero y sencillo. En proyectos grandes, se prefieren bases de datos vectoriales para almacenar la información, sobretodo en

relación a la IA, donde se deben almacenar y tratar embeddings. Una base de datos vectorial puede almacenar y buscar los embeddings mejor que una BD SQL, que no está pensada para ese sistema. Sin embargo, teniendo en cuenta el tamaño del proyecto y la cantidad de información que se maneja, la mejora de rendimiento entre ambas opciones no resultaría apenas perceptible. Además una base de datos SQL relacional permite trabajar con más facilidad el resto de partes del sistema de la base de datos, gracias a la integridad referencial.

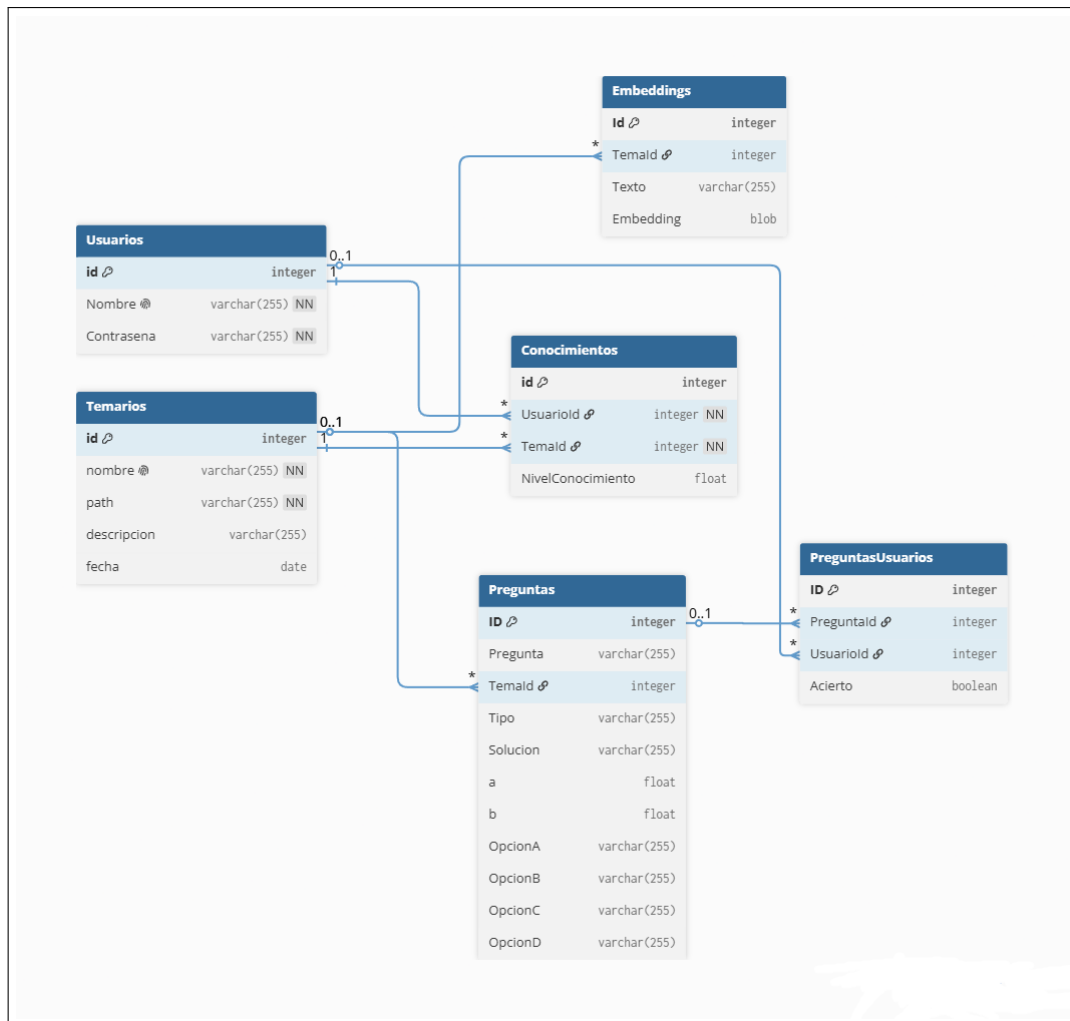


Figura 5.1: Diagrama de la base de datos

La arquitectura de la base de datos usada en el tutor, representada en la figura 5.1, la componen las siguientes tablas:

- **Temarios**: Tabla que contiene información relacionada a los temas que enseña el tutor.
- **Embeddings**: Para poder almacenar correctamente los embeddings en una tabla

SQL y no en una de vectores, se usa el tipo "blob" (Binary large Object). Puede almacenar grandes cantidades de información. El campo texto contiene el fragmento de texto relacionado de un tema concreto del que se extrae el embedding.

- Conocimientos: Contiene el nivel que el sistema asigna a un usuario en un tema según su desempeño al responder preguntas. Los pares de `usuarioId` y `temarioId` son únicos
- PreguntasUsuarios: Se usa para saber qué preguntas ha respondido cada usuario y conocer si la ha acertado o fallado. Los pares de `PreguntaId` y `UsuarioId` son únicos
- Usuarios: El sistema de usuarios, aunque simple, es acorde a los objetivos del proyecto. Un usuario puede registrarse o hacer login en su cuenta, con la que se realiza el seguimiento del nivel de cada usuario.
- Preguntas: Tabla que contiene tanto preguntas de desarrollo como de tipo test que genera el modelo en su interacción con el usuario. Se ha decidido usar el patrón "Single Table Inheritance" debido al parecido de los atributos de ambos tipos de preguntas. Para evitar incoherencias, se han añadido check constraint que comprueban que si el tipo de pregunta es test los valores de opción A,B,C y D no sean NULL y viceversa.

5.1.2 Librerías

Otros paquetes y librerías instalados:

- **Chainlit:** *Chainlit: Documentation* (s.f.) Framework de python capaz de generar interfaces de chat para LLM de forma sencilla, sin necesidad de que el programador cree un frontend. Gracias a los decoradores, como `@cl.on_message`, que incluye el framework, lee los mensajes que el usuario escribe en el chat y envía los textos por parte del LLM. Cuenta con funciones especiales para realizar preguntas, en las que, se envía un mensaje al usuario y se paraliza la ejecución de la función hasta recibir respuesta por parte del usuario. También permite generar botones personalizados dentro de los mensajes, que se ha aprovechado para realizar las preguntas tipo test. También permite gestionar el sistema de login de usuarios de forma sencilla, además guarda la sesión del usuario, por lo que no es necesario iniciar sesión cada vez que se inicia la app. Para esta funcionalidad, necesita una clave secreta, que se debe guardar en el archivo `.env` del programa. Chainlit permite generar fácilmente por consola claves con el comando: `chainlit create-secret`
 - **pyTorch:** Dependencias que necesita VLLM para funcionar correctamente. Tras recibir el mensaje y ser tokenizado (convertir las cadenas de palabras a números), pyTorch ejecuta en GPU gracias a CUDA operaciones matemáticas enormes, calculando cual es el siguiente token más probable que deba generar para dar una respuesta coherente. Esto se realiza token a token, una vez calculado, se descodifica y envía por streaming al usuario.
 - **pypdf:** Librería para leer ficheros pdf. Usado para leer los materiales docentes y generar los embeddings.
-

- **sentence_transformers**: Framework de python capaz de generar embeddings de textos, videos o imágenes con modelos previamente entrenados

5.2 Ingeniería de prompts

Debido a que toda la aplicación se fundamenta en las respuestas generadas por el LLM, la comunicación con el mismo es por tanto una parte vital. La ingeniería de prompts son técnicas, buenas prácticas y recomendaciones para escribir los prompts, las peticiones que un usuario realiza al modelo. Se han estudiado y aplicado al trabajo para optimizar las respuestas generadas, extrayendo la información de *Prompt Engineering Guide* (s.f.).

5.2.1 Estructura y componentes de un prompt

Al enviar mensajes al LLM, los mensajes se envían dentro de una lista de diccionarios. Los diccionarios deben tener los atributos "role" y "content". Hay tres posibles roles:

- **System**: Define la personalidad y tono que debe emplear el LLM
- **User**: Representa al usuario, es donde se introducen los prompts o consultas que realiza
- **Assistant**: Representa la respuesta del LLM. Se usa para mostrar las respuestas obtenidas dentro del historial de la conversación.

Ejemplo:

```
mensaje = [  
{"role": "system" , "content": "Eres un sistema de ayuda y asistencia en la  
educación primaria orientado a hablar con niños"},  
{"role" : "user" , "content": " Hola" },  
{"role" : "assistant", "content": "Hola, ¿Qué quieres aprender hoy?" },  
{"role": "user", "content": "Explicame los pasos que tengo que seguir para  
hacer una división" } ]
```

Gracias a este sistema podemos enviar un historial de mensajes recientes al LLM si se considera necesario, aportando coherencia a la respuesta generada pues puede comprobar los mensajes anteriores.

En lo que respecta al prompt en sí, también se puede dividir en varias secciones. Según la necesidad y requisitos de la tarea que deba cumplir deberemos usar las siguientes partes:

- **Instrucciones**: Ordenes y restricciones que el modelo debe seguir.
 - **Contexto**: Información adicional y externa que ayuda al modelo a generar mejores respuestas
 - **Input data**: La petición realizada por el usuario que debe responder el modelo
 - **Output Indicator**: Especifica el tipo de formato de la salida.
-

5.2.2 Principios fundamentales de diseño

Antes de explicar los distintos tipos de estrategias de prompting, es conveniente explicar algunos de los conceptos y recomendaciones que se aconsejan seguir para escribir buenos prompts.

- **Claridad:** Ser conciso, específico y usar palabras clave al indicar las ordenes o tarea que debe seguir el modelo. Debe evitarse ser impreciso para que el modelo no tenga margen de generar como quiera la respuesta.

Además para indicar claramente el contexto e instrucciones es aconsejable marcarlos con un separador claro, como "hashtags".

Ejemplo:

```
### Instruction ###  
Translate the text below to Spanish:  
Text: "hello!"
```

- **Detallar el prompt:** Es importante ser descriptivo y detallado, sobretodo cuando buscamos un estilo, o formato de salida concretos. Aunque incluir demasiada información tampoco es una buena práctica, debe tenerse en cuenta la relevancia y como va a contribuir a mejorar la respuesta.
- **Evitar indicar que no se debe hacer:** Es mejor indicar de forma clara que pasos o restricciones debe cumplir, en lugar de decir al modelo que no debe de hacer. Se debe evitar situaciones ambiguas en las que el modelo tenga que elegir que hacer si un usuario da contra indicaciones.

Ejemplo:

```
Contexto: Eres un agente que elige películas para sus usuarios. NO  
PREGUNTES POR DATOS NI INTERESES PERSONALES.
```

```
Usuario: Elige una pregunta en base a mis intereses.
```

```
Modelo: Claro, dime que tipo de películas te gustan.
```

Cambiando el prompt por otro donde le indicamos que debe hacer en lugar de que no debe hacer, obtenemos una respuesta mucho más cercana al objetivo de la tarea.

```
Contexto: Eres un agente que elige películas para sus usuarios.  
Instrucciones: Abstente de realizar preguntas personales sobre los  
intereses de los usuarios.
```

```
Usuario: Elige una pregunta en base a mis intereses.
```

```
Modelo: No tengo conocimiento sobre tus gustos, pero aquí tienes una  
lista de las películas más populares actuales.
```

5.2.3 Técnicas y estrategias de interacción

Existen muchas técnicas distintas de prompting, se han estudiado las más relevantes y aplicado algunas de ellas al prototipo. Se citan algunas de las más relevantes:

- **Zero-shot Prompting:** Dar al modelo una instrucción directa sin ejemplos, de forma que se base en su entrenamiento general para generar la respuesta.
- **Few-shot:** Consiste en incluir ejemplos relacionados con la consulta para que el modelo tenga una guía sobre como actuar, es útil para tareas más complejas en las que zero-shot no es capaz de responder como se espera. Los ejemplos proveen conocimiento que dirige el modelo hacia mejores respuestas.
- **Chain of thought:** (CoT) Se pide al modelo realizar las tareas de forma razonada, paso a paso. Puede combinarse con few-shot para obtener mejores resultados, mostrando un proceso de pensamiento paso a paso de ejemplo. O bien puede realizarse sin incluir ejemplos pidiéndole únicamente que piense las cosas paso a paso. Un problema de CoT con Few-shots surge al realizar de forma manual los ejemplos en los que hay que ir paso a paso, en los que puede introducirse algún fallo por error. Para resolver esto, surge Auto-CoT. Se parte de un dataset de preguntas de ejemplo que se pasan a una IA para que genere las respuestas paso a paso, se incluyen en el prompt principal, con el que el modelo responde a la duda del usuario. La figura 5.2 muestra un ejemplo de como funciona AutoCot. Un primer LLM resuelve las preguntas de ejemplo y se introducen en el prompt a enviar junto con la pregunta real que queremos que razone paso a paso.

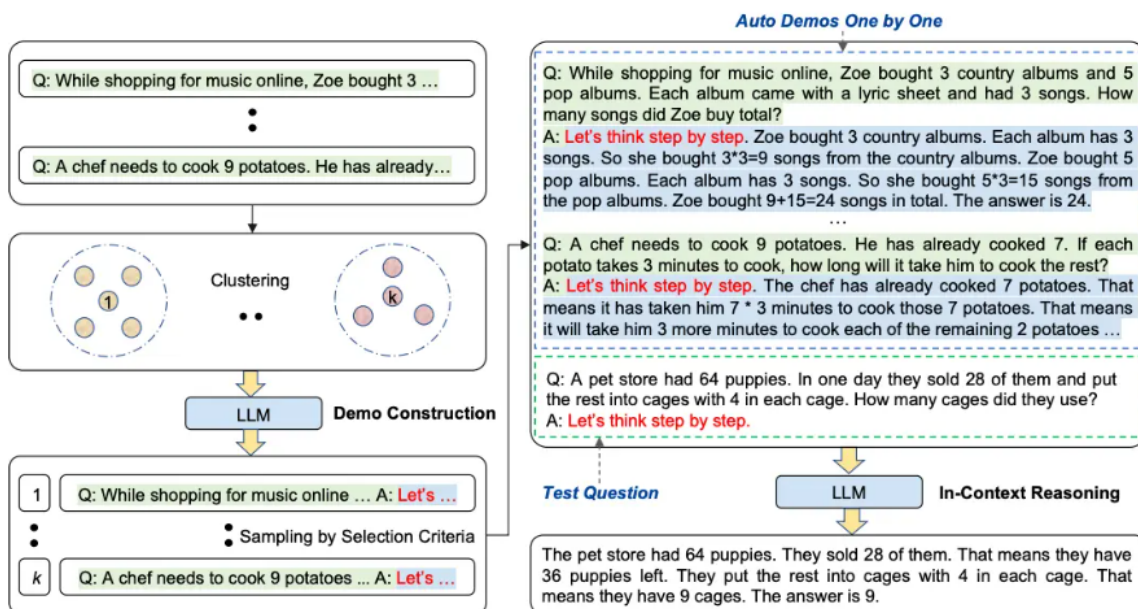


Figura 5.2: Proceso de generación de preguntas usando AutoCot

- **Self-Consistency** : Se trata de usar CoT con few-shots pero generando varias respues-

tas y quedarse con la mejor, permite reducir los errores, especialmente útil en aritmética y razonamientos lógicos.

- **Meta Prompting:** Técnica avanzada de prompting, se centra en mejorar los aspectos estructurales y sintácticos del prompt más que en detalles. Da al modelo una guía por pasos de como estructurar su respuesta. A parte de esta definición, también se suele denominar "Meta prompting" al proceso de pedir a una IA que cree o mejore un prompt, normalmente, un modelo de mayor complejidad optimiza prompts para que lo use un modelo menos inteligente, reduciendo la dependencia con el ingeniero.

Para los prompts usados en el proyecto se ha decidido estructurarlos de la siguiente forma: Primero se le indica al LLM su rol y labor que debe cumplir y se complementa con el contexto e información externa extraída de los temarios. Tras lo que, se indican instrucciones y restricciones concretas que debe seguir, y el formato de salida que debe seguir en caso de necesitarse. Cada parte está separada debidamente y se explica de forma clara y concisa, evitando dar instrucciones negativas o dar demasiada información que pueda acabar por confundir al LLM. Por último, se incluye en los prompts una lista de ejemplos (Few-shot).

6 Desarrollo

El desarrollo de la aplicación se realizó de forma incremental por funcionalidades separadas. El objetivo era tener un prototipo mínimo viable con el que poder interactuar de forma temprana en el desarrollo.

Respecto al prototipo creado, es importante señalar que constituye una prueba de concepto. Con el fin de centrarse en los aspectos clave del estudio, no se han abordado cuestiones fundamentales para un entorno de producción real, como la seguridad del sistema o la gestión de la concurrencia entre múltiples usuarios usando la aplicación de forma simultánea. En este apartado se explicarán los pasos seguidos en la creación del tutor y todas las funcionalidades de las que dispone.

Tras un proceso inicial de investigación del estado del arte y preparar el entorno, donde se hicieron pruebas iniciales, aprendiendo sobre enviar mensajes al modelo, se desarrollaron las cuatro principales funciones que realiza el modelo, que suponen cuatro formas distintas de interactuar con el. Además, existen otras funciones secundarias, como el modo de repasar un tema, o ver el seguimiento del alumno. El funcionamiento de la aplicación se puede ver en la figura 6.1

- **Modo normal:** Modo de comunicación básico, donde los mensajes se envían al LLM sin ninguna orden o instrucción especial, opción para comunicaciones informales.
- **Preguntar:** Se realiza al usuario un cuestionario de preguntas, de desarrollo o test, en base a su nivel de conocimiento en el tema concreto que se esté evaluando. Estas preguntas sirven para actualizar el conocimiento que se tiene del usuario.
- **Resumir:** Extraer los conocimientos principales del tema que solicite el usuario y explicarlos al usuario.
- **Responder a una duda:** Si el usuario pregunta una duda sobre el temario académico, mediante RAG se extraen datos relacionados de los materiales docentes para mejorar el conocimiento del modelo sobre el tema y responder de forma más detallada.

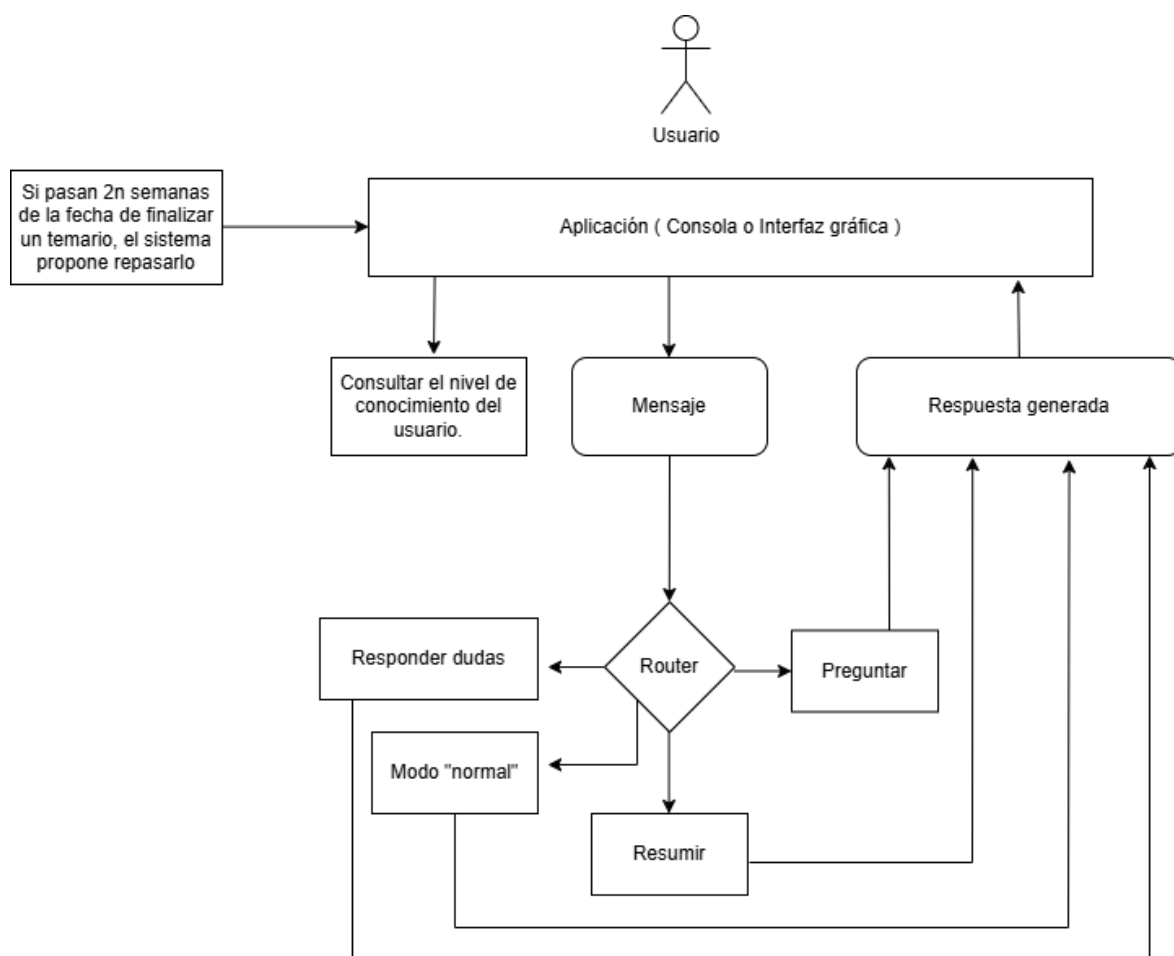


Figura 6.1: Diagrama principal del flujo de la aplicación

6.1 Funcionamiento básico del sistema

Al entrar el usuario a la aplicación, se inicializa una variable local llamada historial, que contendrá los mensajes que se envían al LLM. Se inicializa con el primer mensaje siendo contexto general que debe seguir el modelo, esto se indica gracias a que se marca con el rol system. El objetivo es que el modelo tenga contexto de la conversación para que genere respuestas con un mayor grado de coherencia respecto a los anteriores mensajes. Debido a las limitaciones de tokens que pueden enviarse al modelo, cada varios mensajes se borran los más antiguos que almacena, a excepción del primero de todos que contiene el contexto de system. También se hacen preguntas enviando solo el prompt y no guardando el historial, hay peticiones que es preferible no registrar en el historial para mantener el historial limpio, por ejemplo, las peticiones del enrutador del sistema.

Para enviar los mensajes al LLM se usa httpx, que permite comunicarse con el servidor.

Para realizar llamadas correctas hay muchos parámetros que se pueden introducir según la necesidad de la aplicación, en este caso se han usado los siguientes:

- **Temperatura:** valor, comprendido entre 0 y 1. Cuanto más cercano a 1, las respuestas generadas serán más creativas, cuanto más cercano a 0 son más exactas. Como buscamos responder dudas académicas no hay necesidad de que sea creativo y se le ha asignado 0.3.
- **Max_tokens:** Número máximo de tokens que el modelo puede generar en su respuesta. Es un parámetro importante, establecer un rango máximo genera respuestas más coherentes y reduce la probabilidad de una sobrecarga en el servidor. Para el proyecto se ha establecido el máximo en 4042 tokens, para poder generar resúmenes sin problemas.

6.1.1 Router

Cada vez que un usuario escribe un mensaje, pasa por el router, el cual decide a que módulo de los explicados anteriormente debe enviar el mensaje: resumir, preguntar, explicar o el modo normal. Tiene dos formas posibles de tomar la decisión, en primer lugar un sistema determinista de reglas con varios if, en el que si se cumplen ciertas condiciones, da por sentado el módulo que debe responder a la petición. Funciona comprobando si el mensaje tiene ciertas combinaciones de palabras clave, si detecta que contiene palabras como por ejemplo "hazme" o "genera" y además contiene la palabra "pregunta", enruta con el módulo de generar preguntar. Otro caso, si contiene igualmente palabras clave como las anteriores pero en lugar de pregunta, encuentra la palabra "resumen", enrutará al módulo de resumir temarios.

Ejemplo:

Mensaje: "Genera una pregunta del tema 2" -> PREGUNTAR Mensaje: "Haz un resumen del tema 3" -> RESUMIR

Si no se consigue detectar ninguna estructura predefinida, es el LLM el encargado de decidir a donde enviar el mensaje. La petición del usuario se añade a la plantilla del prompt de enrutar y se envía al modelo. El prompt indica al modelo el contexto, que es un asistente de enrutamiento, las reglas que debe seguir y pone ejemplos de respuestas. Hay cuatro posibles salidas para el prompt: NORMAL, RAG_INFO, RAG_EXAM, RAG_RESUME. A lo largo de las pruebas realizadas al sistema, el router ha funcionado correctamente, el LLM se limita a responder una de estas cuatro opciones siempre en todo momento, y la mayoría de las veces la opción escogida es la idónea para responder al usuario. En muy pocas ocasiones el sistema se ha confundido, y ha decidido escoger la opción RAG_INFO, cuando el usuario quería recibir una pregunta, sin embargo, esta alucinación no es lo suficientemente importante como para poner en duda el sistema. Todos los prompts, debido a su tamaño, están guardados en ficheros de texto que se leen en el código.

6.1.2 Seleccionar tema

Independientemente del modo de funcionamiento que se necesite, algo que tienen todos en común (excepto el modo básico o normal) es la necesidad de extraer información de los materiales docentes. Hay una función dedicada a escoger uno de los posibles materiales docentes de la asignatura en base a la petición del usuario. Desde la base de datos se extraen todos los posibles temas, además de una descripción del mismo con palabras clave. Se pide

de nuevo a la IA que elija un tema, en base al prompt del usuario, y se le entrega la lista de temas numerados junto con la descripción. Si la petición es demasiado vaga para que se pueda deducir un temario acorde, el modelo escoge un tema de forma aleatoria, por ejemplo en caso de que el usuario diga "Hazme dos preguntas para repasar".

El uso de la IA para escoger el tema ha resultado la mejor de las opciones valoradas. Junto a esta, se estudió usar un sistema RAG con embeddings (se explica en el siguiente apartado). Esta opción se descartó por requerir una mayor precisión en el dataset de palabras que comparasen la petición y temarios, además de por la posibilidad de sufrir colisiones semánticas. Una colisión semántica sucede cuando dos textos con diferente significado terminan con vectores numéricos similares que puedan llevar a confundirlos. Para evitar necesitar preparar y evaluar el dataset de palabras clave se optó por la opción de que el propio modelo dictaminase que tema escoger.

Alternativamente, al principio del desarrollo se valoró la opción de no distinguir entre temas directamente, debido al poco contexto que se ha escogido para probar la aplicación, no supone una mejora de rendimiento relevante buscar información dentro de un tema determinado, o en todos los temas a la vez. Sin embargo, se decidió que un enfoque por temas, era más acertado para el proyecto y la posibilidad de escalarlo a futuro. Además, facilitó la implementación de otros sistemas posteriores como resumir temas.

La figura 6.2 muestra los pasos que sigue el programa para la generación de respuestas. La función de seleccionar temas aunque se destaca, está dentro de los distintos módulos.

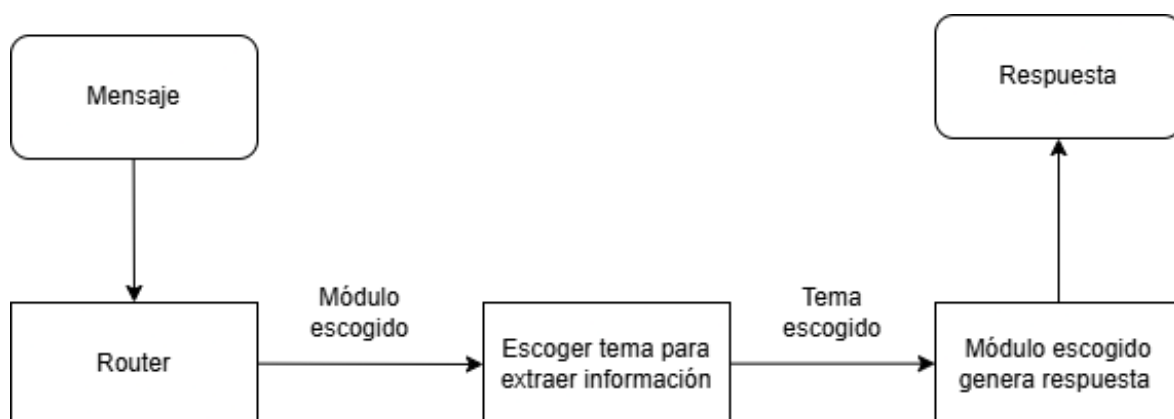


Figura 6.2: Diagrama de pasos que sigue un mensaje para generar una respuesta

6.1.3 Formato en que el modelo devuelve las respuestas

En la gran mayoría de respuestas que da el modelo, debe devolver la salida en un formato concreto, por ejemplo, al seleccionar un tema o en el router, se necesita que devuelva una opción válida pues se tomarán decisiones en base a ese mensaje. Incluir ejemplos siguiendo la metodología de prompts "few-shot", ha resultado útil para estos casos, en los que debe devolver una respuesta simple. Sin embargo, tal y como se explica en los siguientes apartados, algunas de las respuestas que debía devolver eran de mayor complejidad, con varios apartados distintos y podía no respetar los requisitos que se le exigían al formar su salida, en especial al probar la aplicación con modelos de menor nivel. Por este motivo, se decidió usar JSON

como formato de salida.

JSON es un formato estructurado y jerárquico que los modelos IA conocen bien gracias a los entrenamientos que reciben. Hacer que el modelo devuelva las respuestas con este formato mejora la calidad de las respuestas. Sin embargo, por culpa de este mismo entrenamiento, algunas veces, podía incluir caracteres Markdown envolviendo el objeto JSON para generar la respuesta. Los modelos están entrenados, al ser usados en chats, para formar un bloque de código en el que mostrar el JSON, por lo que hubo que hacer una función especial para limpiar las salidas que no tuviesen un formato correcto.

6.2 Responder dudas del usuario

Después de conseguir enviar mensajes básicos a la IA desde el código de forma exitosa, la primera de las funcionalidades principales que se desarrolló fue el modo de responder dudas del usuario. Para responder de forma detallada, obtiene contexto extraído de los materiales docentes, relacionado con la pregunta del usuario, y lo integra en la petición para formar un prompt más detallado. Para obtener esta información usa el sistema RAG (Retrieval Augmented Generation).

El sistema RAG se muestra de forma gráfica en la figura 6.3, los pasos que sigue son los siguientes:

- 1º El usuario hace una consulta que se envía a la función de "modelo de recuperación" (retrieval model).
- 2º El motor del modelo busca información relevante en las fuentes externas.
- 3º Se devuelven los datos relevantes encontrados
- 4º Se combinan, inyectando el contexto aumentado en el mensaje del usuario en un prompt mejorado que se envía al LLM.
- 5º El LLM genera la respuesta que recibe el usuario.

La parte más compleja es el retrieval. La obtención de datos se hace comparando mediante similitud de cosenos los embeddings de la petición del usuario y de los textos de los materiales docentes. Los embeddings son vectores de números que representan texto, por lo que pueden usarse para ver la similitud entre dos textos. En bucle, se realiza la similitud de cosenos con los textos que conforman el tema que se ha escogido, y si el número obtenido sobrepasa un umbral mínimo, se almacena. Tras completar el bucle se escogen los N textos de mayor similitud a la petición. A partir de estos textos escogidos se amplía el contexto seleccionando, además de las líneas de texto de mayor similitud, las siguientes y anteriores a estas para obtener fragmentos de texto coherentes que entregar al LLM.

De forma previa se necesita generar embeddings de los textos. Este proceso consiste en, de forma individual para cada tema, leer el pdf, separando las cadenas de texto en fragmentos que se llaman chunks. No existe un proceso exacto para elegir el tamaño de un chunk, pero se ha decidido que para mantener la consistencia lógica de las frases, en este proyecto el tamaño de un chunk sea el de una línea de texto hasta llegar a un salto de línea. Una vez se tiene el chunk de texto se convierte a un embedding de números gracias a la librería sentence

transformers de python, que permite hacer la operación. Los embedding se almacenan en la base de datos junto al texto que codifican, a la espera de usarse. Esta misma librería es la que necesita usarse para convertir a embeddings la petición del usuario y hacer así la comparación.

La generación de embeddings de los materiales docentes sólo puede realizarse desde la aplicación en modo consola, no es posible realizarlo desde la interfaz gráfica. No existe como tal un rol de administrador en la aplicación, pues la gestión de roles no era uno de los objetivos que se buscaban con este trabajo, pero el modo consola podría verse como una opción exclusiva para desarrolladores del sistema.

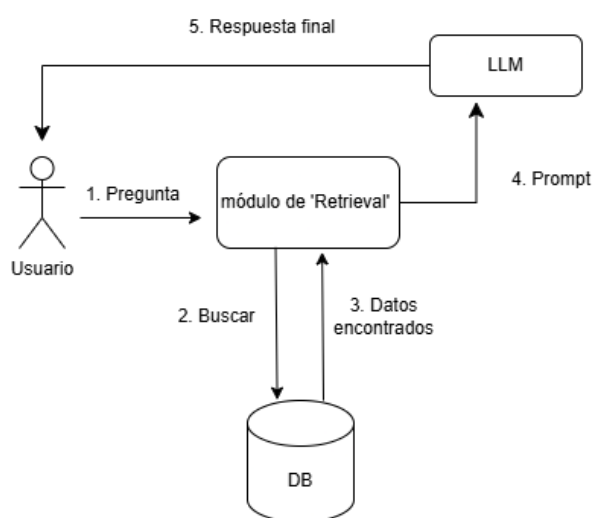


Figura 6.3: Esquema de funcionamiento de RAG

6.3 Resumir contenidos

La función de generar resúmenes es más simple, ante la petición de un usuario, se entrega al modelo directamente todos los textos del tema escogido junto a algunas instrucciones generales, como que sea conciso y extraiga las ideas principales. Es necesario entregar todo el contenido del tema sin usar RAG para que pueda leerlo el modelo y así generar un resumen correcto. El número de tokens de un tema no excede el máximo número de tokens del LLM, y estas operaciones se excluyen del historial para evitar una sobrecarga de tokens en futuros mensajes. Debido a que es un prototipo, sólo se ha probado para los temarios de una única asignatura, y como se ha explicado es suficiente para este caso. Si la aplicación se expandiese a temas de mayor cantidad de texto, se necesitaría de mejores servidores, o un LLM más potente con ventanas de contexto más grandes.

6.4 Realizar preguntas

El sistema tiene la capacidad de generar preguntas adaptadas al nivel del alumno. Existe un valor numérico que mide el nivel de conocimiento de cada usuario para cada tema. Se supone un valor inicial 0, el cual se va actualizando, sumando o restando un valor, según si

el usuario acierta o falla la pregunta. Existen dos tipos de preguntas que pueden darse, de tipo test, donde se plantea la pregunta junto con cuatro posibles opciones al usuario y debe escoger la opción correcta, o preguntas de desarrollo, en las que el usuario debe desarrollar en unas pocas líneas una respuesta.

El funcionamiento es simple, cuando el usuario pide practicar algún tema, en primer lugar se decide cuantas preguntas se le formulan. Al igual que en el router, si se cumplen una serie de reglas, se determina el número de preguntas que debe hacerse. En este caso, si se encuentra la estructura dentro del prompt "número + preguntas o pregunta", directamente se establece ese número como el total de preguntas a realizar. Si no se encuentra la estructura, por ejemplo, en caso de que el usuario escriba el número en formato de texto, se pasa un prompt al modelo con la petición del usuario e instrucciones de como proceder. Si el usuario no especifica un número, por ejemplo, simplemente envía una petición vaga que diga: "hazme preguntas", se escoge el número por defecto que es tres.

Ejemplo de como se establece el número de preguntas:

Petición: Hazme 2 preguntas del tema 4 --> se establece ese dos como número de preguntas

Petición: Hazme cinco preguntas para estudiar el examen --> El modelo lee el prompt y devuelve 5.

Petición: Hazme unas preguntas del tema 2 --> El modelo lee el prompt y devuelve el número por defecto: 3

Una vez establecido el número, se decide el tipo de preguntas. Si el usuario pide que sean de test o de desarrollo, que se comprueba recorriendo la petición y comprobando si aparecen estas palabras, todas serán del tipo especificado. Si no lo indica, antes de empezar cada pregunta se elige una de las dos opciones de forma aleatoria.

Con estos datos, se inicia un bucle for para generar y evaluar preguntas. Al generar una pregunta, primero se selecciona el tema de la pregunta. Originalmente esto se realizaba fuera del bucle for, pero incluirlo dentro permite generar preguntas de varios temas, a menos que el usuario especifique que quiere preguntas de un tema concreto.

6.4.1 Obtención de contexto para generar la pregunta

Tras seleccionar un tema, se obtiene el contexto extraído de los materiales docentes para generar la pregunta. Para ello, primero obtiene la descripción del tema seleccionado de la base de datos. Estas contienen los principales conceptos de los temarios separados por comas. La descripción se convierte a una lista y se escoge uno de los conceptos del tema al azar, la pregunta se generará en base a ese concepto. Para expandir la información se usa el sistema RAG creado para responder a las dudas del usuario, pasando el concepto elegido, de forma que devuelve mucha más información sobre el mismo. Esta información se añade al prompt para generar la pregunta.

6.4.2 Conocimiento y niveles de Bloom

Además, obtiene desde la base de datos el nivel de conocimiento registrado del usuario para un tema (θ), y en base a este se establece un nivel de la taxonomía de Bloom para generar

la pregunta. La taxonomía de Bloom (Armstrong (2016)) clasifica los procesos cognitivos que intervienen en el aprendizaje en niveles jerárquicos, del más simple al más complejo, representados en la tabla 6.1.

Nivel	Verbo principal	Tipo de conocimiento
1. Recordar	Identificar, listar, definir	Memoria literal, hechos, datos
2. Comprender	Explicar, interpretar, resumir	Entender el significado y propósito
3. Aplicar	Usar, ejecutar, resolver	Transferir el conocimiento a situaciones nuevas
4. Analizar	Comparar, distinguir, inferir	Descomponer ideas y relaciones
5. Evaluar	Justificar, argumentar, valorar	Emitir juicios informados
6. Crear	Diseñar, construir, formular	Generar ideas o soluciones originales

Cuadro 6.1: Tabla de niveles taxonómicos de Bloom

Según el nivel de conocimiento del alumno, se pide al LLM que use uno de los niveles, del 1 al 4 de la tabla 6.1. Conforme avance en un tema y responda bien a más preguntas, se le permite ir subiendo en los niveles de Bloom para comprobar cuanto sabe realmente. Los niveles 5 y 6 no se incluyen por la dificultad de adaptarse a preguntas tipo test. Los mejores niveles para generar preguntas son los 2,3 y 4. Estos niveles son clave porque favorecen la metacognición, al requerir comprensión y razonar, mientras que el nivel uno simplemente pide recordar, y esta clase de preguntas pueden resultar obvias. Las preguntas analíticas, de nivel cuatro, permiten calibrar el juicio, al ser en apariencia ambiguas y obligan a comparar entre conceptos similares. Los niveles de bloom, aunque se usan para generar los dos tipos de preguntas, son especialmente útiles para generar preguntas tipo test, ante opciones cerradas, en las que no hay que desarrollar, es más sencillo poder generar preguntas que pidan distinguir conceptos, o poner en situaciones nuevas el conocimiento previamente aprendido. En la implementación dentro del código se han establecido umbrales, representados en la tabla 6.2, que permiten avanzar al siguiente nivel de Bloom. Estos umbrales se han elegido intentando distribuir de forma equitativa a lo largo de lo que sería una curva de aprendizaje normal para un usuario, pero ante una supuesta publicación de la aplicación, convendría estudiar y actualizar estos valores.

Nivel Bloom	Umbral para avanzar al siguiente nivel
1. Recordar	0.5
2. Comprender	1.5
3. Aplicar	2.5
4. Analizar	-

Cuadro 6.2: Umbrales según conocimiento para avanzar al siguiente nivel de Bloom en el código de la aplicación

Además de establecer un nivel de Bloom, se usa el conocimiento del alumno sobre el tema para establecer la dificultad de la pregunta. Si es un valor alto, realiza preguntas más complejas que con valores bajos o por debajo de cero. Devuelve junto con la pregunta un valor numérico que representa la dificultad, y otro valor que representa la discriminación de la pregunta, es decir, cuanto diferencia la pregunta entre alumnos que saben sobre el tema y los que no. Una pregunta de baja dificultad en el nivel uno de bloom tiene una baja discriminación, ya que muchos podrán responderla con facilidad aunque no hayan asimilado correctamente los conocimientos del tema. Estos dos valores, discriminación y dificultad (representados por a y b respectivamente) son necesarios para evaluar la respuesta y el algoritmo que realiza este proceso (IRT). La pregunta y los valores a y b, se devuelven en formato JSON.

Ejemplos de generación de preguntas:

```
Ejemplo 1
conocimiento = 0.33 -> Nivel bajo de conocimiento, empleará el nivel 1 de Bloom.
Consulta: "Hemos empezado en clase el tema 4, hazme una pregunta sencilla para comprobar que lo entienda"
Salida en formato JSON que genera el LLM:
{
    "pregunta" : "Di el objetivo de los diseños de caso de prueba con caja negra",
    "a" : "0.6",
    "b" : 0.2
}
```

```
Ejemplo 2:
conocimiento = 1.46 -> Nivel 2 de bloom. Comprender
Consulta: "Estoy repasando para el examen, hazme una pregunta complicada sobre el tema 1"
Salida en formato JSON que genera el LLM:
{
    "pregunta" : "Nombra y explica las fases de ejecución de un proyecto Maven en su ciclo de vida",
    "a": 1.7,
    "b": 2.0
}
```

Ejemplo 3:

conocimiento = 2.65 -> Nivel alto de conocimiento, Nivel 4 de Bloom (Analizar)

Consulta: "Hazme una pregunta difícil sobre técnicas de pruebas del software"

Salida en formato JSON que genera el LLM:

```
{
  "pregunta": "Dado un sistema con múltiples dependencias externas, analiza las
    ventajas e inconvenientes de utilizar mocks frente a stubs en pruebas
    unitarias y cómo afectan a la fiabilidad de los resultados",
  "a": 2.4,
  "b": 2.3
}
```

Los ejemplos mostrados son para preguntas de desarrollo. La generación de una pregunta tipo test sigue la misma lógica, aunque devuelve además cuatro posibles respuestas y la solución correcta.

Ejemplo 1:

Entrada:

Consulta: Hazme una pregunta tipo test sobre el tema 4

Conocimiento : -0.2 -> Conocimiento negativo, el usuario se ha equivocado al responder en preguntas anteriores del tema 4, el modelo debe generar una pregunta fácil con el nivel 1 de Bloom.

Salida:

```
{
  "Pregunta": "¿Cuál es la principal característica de las pruebas de caja negra?",
  "OpcionA": "Analizan la estructura interna del código",
  "OpcionB": "Se centran en la funcionalidad sin considerar la implementación interna",
  "OpcionC": "Evalúan el rendimiento del sistema bajo carga",
  "OpcionD": "Se utilizan únicamente en fases finales del desarrollo",
  "Solucion": "B"
  "a": 0.5,
  "b": 0.1
}
```

6.4.3 Evaluación de respuestas

Una vez generada la pregunta, el alumno la responde y el sistema la evalúa. Si la pregunta es de tipo de desarrollo, el modelo recibe la pregunta y la solución del alumno y decide si es una respuesta correcta para la pregunta. Devuelve, en formato JSON tres campos: Evaluación, que siempre será "CORRECTO" o "FALSO", el motivo de su decisión, por ejemplo "La respuesta del alumno es demasiado simple y no incluye la idea principal" y por último la solución del Modelo para la pregunta. Estos valores se usan para imprimir por pantalla el resultado. Después guarda la pregunta y la respuesta en la base de datos del programa, y además guarda si el usuario la ha acertado o fallado.

Ejemplo de evaluación de una pregunta generada por el LLM:

```
Pregunta: ¿Qué es HTTP?
Respuesta: Es un protocolo

{
  "Evaluacion": "FALSO",
  "Motivo": "La respuesta es demasiado vaga e incompleta.",
  "Solucion": "HTTP es un protocolo de comunicación que permite la transferencia de
    información en la web entre cliente y servidor."
}
```

La evaluación de una pregunta tipo test es bastante más sencilla, pues solo tiene que comprobar que la opción escogida por el usuario fuese la correcta. También guarda la pregunta y si el usuario la acierta o la falla.

6.4.4 Actualizar el nivel de conocimiento del usuario con IRT

Una vez el usuario acierta o falla la pregunta, esto se debe ver reflejado en el nivel de conocimiento que el sistema tiene de él. En preguntas de desarrollo se usa IRT 2PL. IRT (Item response theory, *Two-Parameter Item Response Theory (2PL)* (2026)) es un algoritmo psicométrico para calcular la probabilidad de acertar una pregunta por parte de un estudiante según su conocimiento sobre el tema. Concretamente, IRT 2PL usa dos variables además del conocimiento, dificultad de la pregunta y su discriminación, valores obtenidos al generar la pregunta. La figura 6.4 muestra como calcular la probabilidad de acierto.

$$P(\theta) = \frac{1}{1 + e^{-a(\theta-b)}}$$

Figura 6.4: Función IRT 2PL para el cálculo de probabilidad de acierto

A medida que el nivel de conocimiento (θ) crece, la probabilidad de acertar aumenta, con inflexión en $\theta = b$, formando una curva sigmoideal

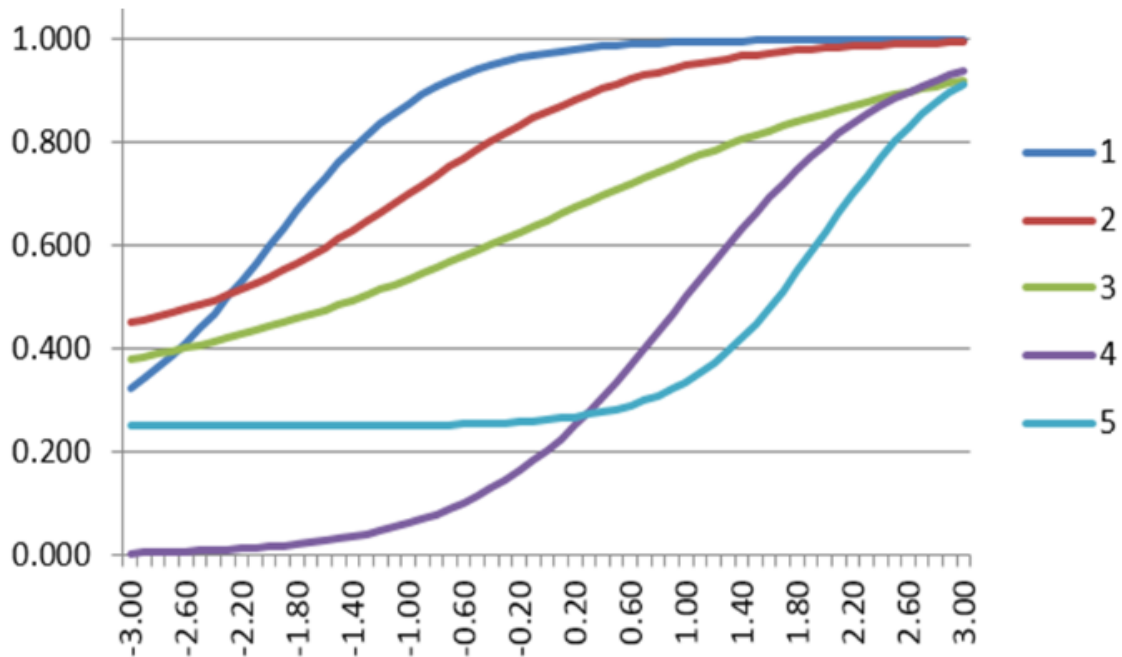


Figura 6.5: Gráfica de $P(\theta)$ en IRT 1PL

La gráfica de la figura 6.5 muestra la evolución de la probabilidad de acierto (eje Y) para distintos valores de conocimiento (eje X), cada línea tiene un valor distinto de dificultad constante y se presupone un mismo nivel común de discriminación igual a uno en todas (el modelo IRT 1PL).

Gracias a esto podemos actualizar el conocimiento, de forma inteligente, fallar una pregunta con una alta probabilidad de ser acertada penaliza más que una fácil, y acertarla supone también una mayor recompensa. Para actualizar el conocimiento se usa la fórmula ERS (Elo Rating System) adaptada a IRT, propuesta por Pelánek en 2016, obtenida de Vermeiren y cols. (2025).

$$\theta_{t+1} = \theta_t + K \cdot a \cdot (u - P(\theta_t))$$

Figura 6.6: Actualización del conocimiento del alumno mediante ERS-IRT (Pelánek, 2016)

En la fórmula de actualización del conocimiento, de la figura 6.6, u representa el valor de la respuesta del alumno, si ha acertado es 1, si ha fallado es 0. A este valor se le resta la probabilidad de acertar la pregunta y es multiplicado por la discriminación del ítem y la tasa de aprendizaje, representada por k . k sería la pendiente de la curva, representa la sensibilidad del sistema o cuán rápido se actualiza, a mayores valores de k , mayor la actualización. En las preguntas de desarrollo del programa es un valor constante de 0.5. De nuevo, cabría estudiar su uso con usuarios reales e ir calibrando su valor hasta llegar a un punto óptimo.

Este valor obtenido se suma al nivel anterior de conocimiento y obtenemos el nuevo valor de conocimiento, que se registra en la base de datos y se usa en la siguiente pregunta del tema.

Las preguntas tipo test suponen un problema, se pueden acertar por azar ya que se debe elegir una opción. Por ello se introduce un tercer componente, c que representa este factor. Se usa en IRT 3PL y su formula es la de la figura 6.7:

$$P(\theta) = c + (1 - c) \cdot \frac{1}{1 + e^{-a(\theta-b)}}$$

Figura 6.7: Función IRT 3PL para el cálculo de probabilidad de acierto

En el prototipo, como genera preguntas tipo test de 4 opciones se ha decidido que c sea una constante de 0.25.

Este sistema sería totalmente útil y robusto, pero los sistemas tipo test tradicionales presentan problemas que pueden mejorarse mediante CBM.

6.4.5 Test CBM

A nivel pedagógico, un test tradicional no mide si realmente sabe la respuesta, aquel que acierte de casualidad obtiene el mismo resultado que un alumno que domine el tema. En un examen tipo test, un alumno puede arriesgarse a acertar una pregunta que no sepa descartando opciones poco probables, mientras que un alumno que sea más inseguro y prudente puede que no responda a la pregunta, quedando desfavorecido y obteniendo una nota similar a alguien con menos conocimiento

El sistema CBM de evaluación (Gardner-Medwin (1990)), se propuso para solucionar esto. Los alumnos además de responder a la pregunta, indican su nivel de confianza en su respuesta, según las investigaciones de Gardner-Medwin, ese cambio estimula un pensamiento más crítico en los alumnos sobre el conocimiento propio.

Se establecen 3 niveles de confianza. No está seguro (1), seguridad media (2), bastante seguro (3). Según el nivel escogido de confianza, tanto la penalización de fallar como la recompensa de acertar aumentan de forma proporcional.

Este sistema tiene varios beneficios:

- El estudiante se piensa más sus propias ideas y que responde, cuestionándose su propio conocimiento.
- Los alumnos inseguros pueden aumentar la confianza en sus conocimientos.
- Es más justo, pues puntúa más una respuesta pensada que una acertada al azar.
- El profesor obtiene una evaluación más fiable en la que identificar que conceptos no han sido bien asimilados por la clase.

Existen fórmulas para calcular notas de exámenes test basándose en CBM, sin embargo, teniendo un sistema IRT creado que actualiza el nivel de conocimiento de forma dinámica tras cada pregunta, esto no era del todo compatible. Se ha adaptado al sistema al cambiar el

valor k , dentro de la formula ERS para actualizar el conocimiento del alumno. En preguntas de desarrollo este valor, que representa la pendiente de la curva, es constante, sin embargo en las preguntas tipo test varía según el nivel de confianza depositado en la respuesta escogida, de forma que con una confianza alta, la actualización del conocimiento será mayor que si se escogiese una confianza menor o intermedia. Este sistema combina los beneficios pedagógicos de realizar preguntas tipo test CBM con el cálculo de conocimiento del alumno en base a IRT.

Finalmente, cuando se han completado todas las preguntas y termina el bucle for, el programa muestra el número de preguntas acertadas sobre el total, y retoma el modo de chat conversacional normal. La figura 6.8 muestra de forma gráfica todo el proceso de generar preguntas.

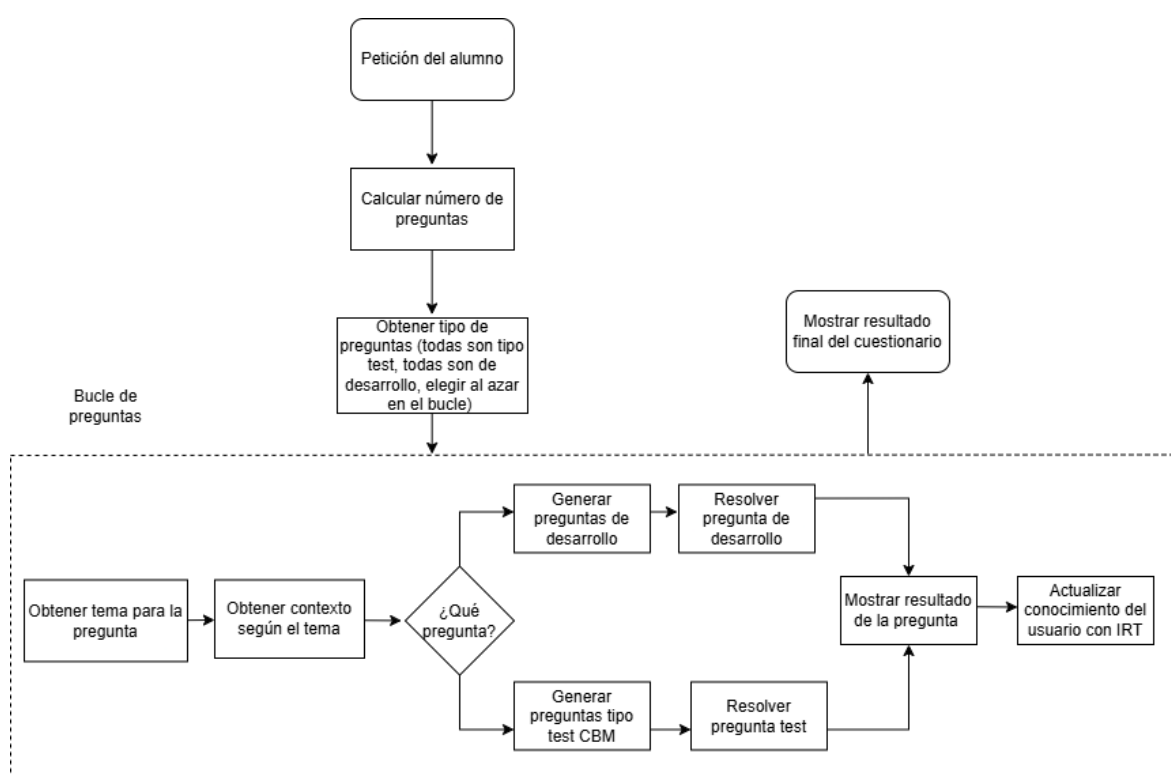


Figura 6.8: Diagrama del funcionamiento del sistema de preguntas

6.4.6 Generar preguntas mediante IA y Presets de preguntas

Al generar preguntas mediante modelos de IA, como hace el programa, nos enfrentamos a una serie de ventajas y desventajas, que no encontramos en un sistema con bancos de preguntas predeterminadas. Por un lado, podemos obtener preguntas personalizadas según los criterios y requisitos pedidos en el mensaje del alumno, el banco disponible de preguntas no tiene límite, y por tanto se elimina el riesgo de plantear al alumno preguntas repetidas, además delegar el trabajo de crear preguntas a la IA, elimina dependencias con el ingeniero y posibles sesgos o falta de conocimiento para plantear las preguntas. Sin embargo, siempre existe el

riesgo de que la IA introduzca errores o ambigüedades en sus mensajes, o que no asigne los niveles de dificultad de forma correcta a las preguntas. Para un correcto funcionamiento del sistema, estas preguntas deberían ser revisadas por docentes, que aseguren que las preguntas generadas por la IA son válidas.

6.5 Recordar temas

A nivel pedagógico, es útil y se recomienda repasar conceptos aprendidos cada dos semanas para no olvidarlos. Esto se ha trasladado a la aplicación.

Al iniciar el programa, cada dos semanas desde la fecha en la que se finalice un temario, se recomienda repasar el tema en cuestión. Por ejemplo, si un usuario entra un día en el que se cumplen seis semanas de finalizar un tema y dos semanas de finalizar otro tema, propondrá repasar ambos temas. Si el usuario acepta, recopila todas las preguntas que realizó del tema anteriormente y vuelve a plantearlas.

En la base de datos se almacenan las respuestas que generó el LLM para cada pregunta. Por lo que, para evaluar las respuestas del alumno en las preguntas de desarrollo, basta con pedirle al LLM que compare ambas soluciones y determine si la del alumno es correcta en base a la respuesta correcta guardada de referencia.

Una vez finaliza el bucle de preguntas muestra cuantas preguntas ha acertado y fallado.

Este sistema supone un problema para el modelo de evaluación IRT, al calcular la probabilidad de acierto, da por sentado que no conoce la respuesta ni ha visto antes la pregunta. El objetivo es recordar los conocimientos aprendidos en días anteriores mediante las preguntas que ha hecho, pero debe realizarse con precaución pues se podría sobre valorar el conocimiento del alumno. Además, actualiza la base de datos, si el alumno había fallado una pregunta originalmente, que ahora acierta, la marca como acertada, y viceversa.

Ejemplo de como se muestra desde consola esta opción al iniciar el chat:

```
-----  
CHAT CON QWEN  
  
Para volver al menú pulse 0  
-----  
Hoy, se cumplen 2 semanas desde que en clase se terminó de estudiar los temas: tema  
5 ppss. ¿Desea repasar estos temarios?. (Escriba S o N)
```

Figura 6.9: Repasar temarios, visto desde la consola

6.6 Interfaz gráfica

Tras completar el funcionamiento base de la aplicación se pasó a trabajar para darle una interfaz gráfica. Originalmente se pensaba crear un Frontend típico, basado en JS, sin embargo este método suponía un sobre esfuerzo innecesario para los objetivos del proyecto. Tras

investigar sobre distintas opciones se descubrió Chainlit, framework para generar interfaces de forma sencilla. Está ideado para generar interfaces gráficas simples estilo chat para presentar prototipos sin necesidad de desarrollar un Frontend, por lo que esta opción era ideal para el proyecto. A nivel de desarrollo, fue necesario cambiar las funciones síncronas a asíncronas, con lo cual, se pudo reusar gran parte del código. Aparte de esto, tuvieron que adaptarse todos los mensajes que se imprimían por consola para ser enviados a la interfaz mediante las funciones que incluye la librería de chainlit. Chainlit también permite que el modelo realice preguntas gracias a una función especial que detiene el flujo esperando que el alumno responda.

Además genera la interfaz para el sistema de login y simplifica la lógica de acceso, aparte de la interfaz del chat, por lo que ese aspecto está cubierto.

Pueden incluirse botones especiales en el chat, llamados Actions, al pulsarse, se dispara el evento correspondiente al botón. Esta función se ha aprovechado para que los alumnos puedan consultar su nivel. Las botones "Actions" pueden incluirse en preguntas, por lo que también se han aprovechado para representar las opciones en las preguntas tipo test, permitiendo una interfaz más limpia y ordenada a nivel visual.

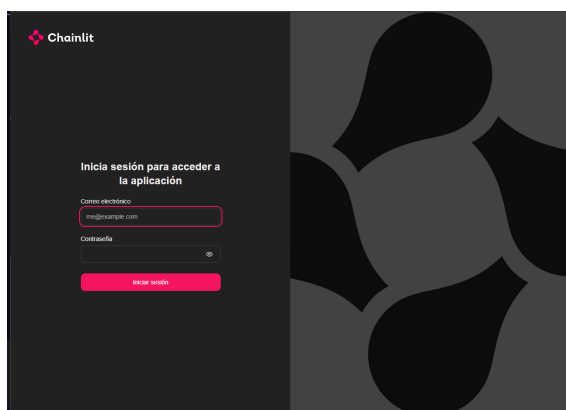


Figura 6.10: Pantalla de login de la aplicación con Chainlit

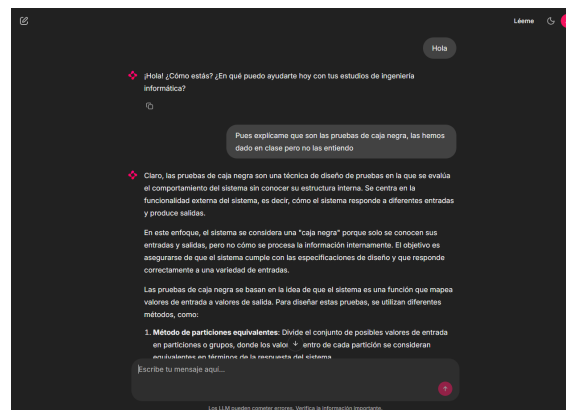


Figura 6.11: Pantalla principal de mensajes con Chainlit

Por otro lado, desde consola, se puede acceder a características no disponibles en la interfaz gráfica. Estas son opciones secundarias y reservadas para administradores del sistema, aunque la aplicación no cuenta con roles de usuarios por ser un prototipo centrado en la parte del chat con el tutor. Es posible generar nuevos embeddings, registrar un nuevo tema en la base de datos, o actualizar la descripción del tema. Las descripciones de los temarios son muy importantes para RAG, razón por la que se añadió una opción para cambiarlas desde el menú.

Además, permite consultar el progreso del alumno en todos los temas que haya estudiado, tanto por consola como por la interfaz. Puede comprobar el nivel de conocimiento registrado y cuantas preguntas ha fallado y acertado por cada tema. En el modo consola, se encuentra fuera del chat dentro del menú principal.

```
Introduzca su usuario: (para registrar un nuevo usuario pulse la tecla 0)
Ana
Introduzca su contraseña:
123

#####
                MENÚ PRINCIPAL
#####
1. Hablar con qwen
2. Comprobar tu nivel
3. Generar nuevos embeddings
4. Añadir un nuevo tema a la base de datos
5. Actualizar descripción de un tema
6. Salir
#####
Seleccione una opcion
```

Figura 6.12: Interfaz de consola de la aplicación

7 Resultados

Una vez finalizado el prototipo, se estudió y midió su uso. En este apartado se muestran métricas obtenidas de estudiar dichas respuestas y cuestionarios realizados por usuarios reales que se han prestado a probar la aplicación. De forma adicional, se han incluido ejemplos de conversaciones con el tutor en el anexo 9.2

7.1 Métricas medidas

7.1.1 Latencia

Tiempo medio de espera para respuestas. Para medirlo, se ha importado al programa la librería time y se ha medido el tiempo de espera, entre que el usuario envía un mensaje, y el LLM imprime por pantalla su respuesta. Este periodo incluye la selección de que módulo debe responder, la elección del tema y la generación de la respuesta en sí misma.

Módulo	Tiempo medio de petición en segundos
Normal	1.79
Resolver dudas	13.37
Resumir contenidos	11.34
Generar preguntas test	4.43
Generar preguntas de desarrollo	3.45
Evaluar preguntas de desarrollo	5.14

Cuadro 7.1: Medición de la latencia para diferentes operaciones en el sistema

Los tiempos mostrados son medias obtenidas tras medir varias respuestas, sin embargo cabe destacar la disparidad entre los tiempos medidos en las peticiones respondidas por el mismo módulo.

Por ejemplo, al pedir resolver dudas, si se limita a responder una duda sencilla en pocas líneas de texto generado, puede tardar menos de diez segundos en generar la respuesta, sin

embargo, operaciones más complejas como pedir que genere ejemplos con código o pedir que represente de forma gráfica la estructura de un proyecto maven, provocan tiempos cercanos a los 20 segundos.

Los mejores tiempos medidos son los del modo de funcionamiento normal o básico. Este resultado es esperable pues son aquellos mensajes que se envían sin pasos previos o requisitos especiales al LLM. Aún así, incluso en este tipo de respuestas se han obtenido tiempos dispares. El LLM puede responder a mensajes simples como "hola" en menos de un segundo, sin embargo otras peticiones informales, como "Cuéntame un chiste de informáticos" cuestan algunos segundos más, elevando la media.

7.1.2 BERTScore

BERTScore, Zhang y cols. (2020), mide la cercanía semántica de una respuesta generada por el ITS a una respuesta ideal de referencia. Convierte a embeddings las palabras del mensaje generado y el texto de referencia, intentando emparejar los tokens de ambos textos que sean más parecidos. Compara los tokens mediante similitud de cosenos. Después aplica tres métricas a los textos:

- **Precision (PBert)**: Mide si cada token de la frase probada tiene pareja en la referencia. Penaliza el contenido extra o de relleno.
- **Recall (RBert)**: Comprueba que los tokens de referencia tengan pareja en la frase probada, comprobando cuanto texto de esta se cubre.
- **Medida "F1" (FBert)**: Media armónica entre PBert y RBert, es un balance entre ambas métricas.

Las fórmulas de cálculo de las métricas son las siguientes, siendo la respuesta de referencia el vector $x = \langle x_1, \dots, x_n \rangle$, y una respuesta a probar $\hat{x} = \langle \hat{x}_1 \dots, \hat{x}_n \rangle$.

$$R_{\text{BERT}} = \frac{1}{|\hat{x}|} \sum_{x_i \in x} \max_{\hat{x}_j \in \hat{x}} x_i^\top \hat{x}_j \quad (7.1)$$

$$P_{\text{BERT}} = \frac{1}{|\hat{x}|} \sum_{\hat{x}_j \in \hat{x}} \max_{x_i \in x} x_i^\top \hat{x}_j \quad (7.2)$$

$$F_{\text{BERT}} = 2 \cdot \frac{P_{\text{BERT}} \cdot R_{\text{BERT}}}{P_{\text{BERT}} + R_{\text{BERT}}} \quad (7.3)$$

Figura 7.1: Funciones de las métricas de BERTScore : Recall (R_{BERT}), Precision (P_{BERT}) y F1 (F_{BERT})

Usar BERTscore para probar el código es sencillo, debe instalarse en el entorno y en el código se le pasan por parámetro a la función correspondiente las frases a comparar, además del idioma. La propia función importada se encarga de aplicar las formulas de la figura 7.1.

7.1.2.1 Primera prueba de BERTscore

- **Petición:** ¿Qué es el método del camino básico ?
- **Respuesta de referencia:**

Es un método de DISEÑO de pruebas de caja blanca que permite ejercitar (ejecutar) cada camino independiente en el programa. Fue propuesto inicialmente por Tom McCabe en 1976. Este método permite al diseñador de casos de prueba obtener una medida de la complejidad lógica de un diseño procedimental y usar esta medida como guía para la definición de un conjunto básico de caminos de ejecución. Estaremos ejecutando TODAS las sentencias del programa, al menos una vez. Además estaremos garantizando que TODAS las condiciones se ejecutan en sus vertientes verdadero/falso. Un camino independiente es un camino en un grafo de flujo (CFG) que difiere de otros caminos en al menos un nuevo conjunto de sentencias y/o una nueva condición (Pressman, 2001)

- **Respuesta generada por el tutor:**

El método del camino básico, también conocido como el método de McCabe, fue propuesto inicialmente por Thomas J. McCabe en 1976. Este método es una técnica de diseño de pruebas de caja blanca que se utiliza para evaluar la complejidad lógica de un programa y para definir un conjunto básico de caminos de ejecución que permiten probar el programa de manera efectiva.

El objetivo principal de este método es minimizar el número de casos de prueba necesarios para detectar el mayor número posible de defectos en el código. Se basa en la construcción de un grafo de flujo de control (CFG) a partir del código fuente del programa. A partir de este grafo, se calcula la complejidad ciclomática del programa, que es una medida de la complejidad lógica del diseño procedimental.

Un camino independiente en este contexto es un camino en el grafo de flujo que difiere de otros caminos en al menos un nuevo conjunto de sentencias y/o una nueva condición. La idea es que al ejecutar todos los caminos independientes, se asegura que todas las sentencias del programa se ejecuten al menos una vez y que todas las condiciones se ejecuten en sus vertientes verdaderas y falsas.

En resumen, el método del camino básico es una técnica que permite al diseñador de pruebas obtener una medida de la complejidad lógica de un programa y usar esta medida como guía para definir un conjunto básico de caminos de ejecución que permiten probar el programa de manera exhaustiva y eficiente.

- **Resultados**
-

Precisión	0.7487
Recall	0.7974
F1	0.7723

Cuadro 7.2: Resultados de la primera prueba con BERTscore

Los datos obtenidos en la primera prueba entran dentro de un rango aceptable y correcto, aún con margen de mejora. No se tiene en consideración ningún límite estricto para estas pruebas, pues son respuestas generadas con cierto grado de libertad por parte del LLM y no traducciones de un idioma a otro que deban ser exactas.

7.1.2.2 Segunda prueba con BERTscore

- **Petición:** ¿Qué es un test parametrizado?
- **Respuesta de referencia:**

Si el código de VARIOS tests es idéntico exceptuando los valores concretos del caso de prueba que cada uno implementa, podemos sustituirlos por UN test parametrizado.

Se trata de implementar UN único test, que anotaremos con `@ParameterizedTest`, y que tendrá como parámetros los valores concretos en los que se diferencian los tests a los que sustituye.

Si el test parametrizado solamente necesita UN parámetro, de tipo primitivo o `String`, usaremos la anotación `@ValueSource` para indicar los valores para ese parámetro.

Si el método anotado con `@ParameterizedTest` requiere parámetros de tipos diferentes, usaremos la anotación `@MethodSource` indicando un nombre de método.

El método `casosDePrueba` devuelve una colección de "tuplas" de valores (cada tupla representa una fila de la tabla de casos de prueba). El número de elementos de la tupla se corresponderá con el número de parámetros del test parametrizado. El test parametrizado se invocará tantas veces como elementos de tipo `Arguments` tengamos.

- **Respuesta generada por el tutor:**

En el contexto de las pruebas unitarias en Java, utilizando frameworks como JUnit 5, puedes implementar tests parametrizados de varias maneras:

1. `@ValueSource`: Esta anotación se utiliza cuando el test requiere un solo parámetro de un tipo primitivo o `String`. Por ejemplo, si estás probando una función que opera con números, puedes usar `@ValueSource(ints = {...})` para proporcionar varios valores enteros.

2. `@MethodSource`: Esta anotación se utiliza cuando el test requiere múltiples parámetros de tipos diferentes, o cuando los datos de entrada son más complejos. En este caso, defines un método que devuelve un `Stream` de objetos `Arguments`, donde cada objeto `Arguments` representa un conjunto de parámetros para el test.
 3. `@CsvSource` o `@CsvFileSource`: Estas anotaciones se utilizan cuando los datos de entrada están en formato CSV. Puedes proporcionar los datos directamente en la anotación o referenciar un archivo CSV.
- Los tests parametrizados son útiles para validar que un método funcione correctamente con una variedad de entradas, lo que ayuda a mejorar la cobertura de pruebas y la robustez del código.

- **Resultados**

Precisión	0.6772
Recall	0.7104
F1	0.6934

Cuadro 7.3: Resultados de la segunda prueba con BERTscore

Los resultados obtenidos en la segunda prueba han sido ligeramente peores a los de la primera, en especial la métrica de precisión, cae por debajo de 0.7, alejándose de una respuesta ideal. En este caso, el tutor ha añadido información adicional no existente en los materiales docentes, nombrando la anotación `@CsvSource`, empeorando las métricas en el proceso. La anotación guarda relación con los tests parametrizados por los que pregunta el usuario, sin embargo si no se nombra en los materiales docentes, en una respuesta ideal no debería nombrarla el tutor, tal grado de precisión es difícil de alcanzar en la actualidad.

La respuesta de referencia y la generada contenían ejemplos de código, que no se han tenido en cuenta en el análisis para evitar que, al ser diferentes, disminuyesen las métricas.

7.1.2.3 Tercera prueba con BERTscore

- **Petición:** Explícame que es un artefacto maven y sus coordenadas.
- **Respuesta de referencia:**

Un ARTEFACTO es cualquier fichero que Maven identifica por sus COORDENADAS, y que Maven descarga, instala o despliega de forma automática.
Ejemplos: ficheros *.pom, *.jar, *.war, *.ear

Las COORDENADAS de un artefacto se indican como: groupId:artifactId:version

groupId: es el identificador de Grupo (<groupId/>)

artifactId: es el identificador del artefacto (nombre del archivo) (<artifactId/>)

version: es la versión del artefacto (<version/>)

packaging. es el empaquetado del artefacto (<packaging/>). Por defecto su valor es jar. No se incluye de forma explícita en las coordenadas (aunque podemos añadirlo al final)

Los artefactos Maven residen en REPOSITARIOS:

remotos (p.ej. <https://mvnrepository.com/>)

locales (p.ej. `\$HOME/.m2/repository/`)

Las coordenadas de un artefacto, no sólo identifican al fichero correspondiente, sino que también nos permiten LOCALIZAR exactamente dónde se encuentra dicho fichero (en un repositorio local o remoto). Por ejemplo:

coordenadas: org.junit.jupiter:junit-jupiter:5.11.4

fichero en el disco duro: `\$HOME/.m2/repository/org/junit/jupiter/junit-jupiter/5.11.4/junit-jupiter-5.11.4.jar`

Cuando maven necesita usar algún artefacto para construir el proyecto, primero comprueba si el fichero correspondiente está almacenado en el repositorio local, y si no, lo descargar del repositorio remoto

- Respuesta generada por el tutor:

Un artefacto Maven es cualquier fichero que Maven identifica y maneja de forma automática. Estos ficheros pueden ser de varios tipos, como `.pom`, `.jar`, `.war`, `.ear`, entre otros. Maven se encarga de descargar, instalar y desplegar estos artefactos según sea necesario para construir y gestionar el proyecto.

Las coordenadas de un artefacto** son una forma única de identificar cada artefacto en Maven. Estas coordenadas se componen de tres elementos principales:

1. `groupId`: Es el identificador de grupo. Generalmente, se utiliza para representar el dominio o el espacio de nombres del proyecto. Por ejemplo, `org.springframework` para los artefactos de Spring Framework.
2. `artifactId`: Es el identificador del artefacto, que es el nombre del archivo. Por ejemplo, `spring-core` para el núcleo de Spring.
3. `version`: Es la versión del artefacto. Por ejemplo, `5.3.10`.

Estas coordenadas se escriben en el siguiente formato:

`groupId:artifactId:version`

Por ejemplo, el artefacto `spring-core` de la versión `5.3.10` de Spring Framework tendría las coordenadas:

`org.springframework:spring-core:5.3.10`

Además, hay un cuarto elemento opcional que no se incluye en las coordenadas pero que puede ser especificado en el archivo `pom.xml`:

4. `packaging`: Es el tipo de empaquetado del artefacto. Por defecto, su valor es `jar`, pero también puede ser `war`, `ear`, entre otros. Este valor no se incluye en las coordenadas, pero se puede especificar en el archivo `pom.xml` si es necesario.

En resumen, las coordenadas de un artefacto Maven son una forma única de identificar y gestionar los ficheros necesarios para un proyecto, permitiendo a Maven descargarlos, instalarlos y desplegarlos de forma automática.

• Resultados

Precisión	0.7378
Recall	0.7199
F1	0.7287

Cuadro 7.4: Resultados de la tercera prueba con BERTscore

Tras las tres pruebas, se puede concluir que las peticiones realizadas en relación al código, se alejan más de las respuestas de referencia (véase la segunda prueba), mientras las peticiones puramente teóricas son más cercanas a los materiales docentes (véase la primera prueba).

7.1.3 LLMs actuando como jueces

Para comprobar si las respuestas son lógicas y coherentes, se han compartido algunas conversaciones con varios modelos de IA generativa, pidiéndoles que actuaran de jueces y dictaminasen si la respuesta era lógica o contenía alucinaciones.

Usar "LLM as judges", Zheng y cols. (2023), es una práctica que ha ganado popularidad recientemente. Algunos modelos como jueces, concretamente los basados en GPT-4, pueden alcanzar un 85% de acuerdo con evaluadores humanos, pero no están exentos de cometer errores y tener sesgos. En lo que afecta a esta evaluación, se debe tener en cuenta el sesgo de verbosidad. Los jueces tienden a preferir respuestas largas y artificiosas aunque no sean mejores que las cortas. También pueden ser engañados por respuestas incorrectas que impliquen razonamientos lógicos o matemáticos, incluso si el modelo pudiera haber resuelto el problema por sí solo dando el razonamiento correcto.

La mejor forma de reducir fallos es usar respuestas de referencia. Algunos modelos pasan de un porcentaje de fallo del 70% al 15%. Diversos expertos aconsejan pedir al juez que genere primero una respuesta a la petición del usuario y luego la compare con la respuesta que se debe evaluar.

A los LLMs se les ha enviado un prompt con las instrucciones que deben seguir para evaluar las respuestas del tutor. Se puede consultar en el anexo 9.1

Con este prompt se han evaluado tres respuestas que el tutor ha dado a las siguientes preguntas:

Pregunta 1	Explícame la estructura de un proyecto Maven
Pregunta 2	Como puedo implementar un driver mediante stubs en mi código
Pregunta 3	¿Qué es un test parametrizado? pon un ejemplo

Cuadro 7.5: Preguntas a evaluar con "LLM as judges"

Respuestas que generó el tutor:

- **Respuesta a la pregunta 1:**

Claro, la estructura de un proyecto Maven estándar se organiza de manera clara y estructurada para facilitar la gestión de los diferentes componentes del proyecto. Aquí te explico los elementos principales de esta estructura:

1. Directorio raíz del proyecto: Este es el directorio principal que contiene todos los archivos y subdirectorios del proyecto Maven.
2. `src/main/java`: Este directorio alberga el código fuente del proyecto escrito en Java. Aquí se encuentran las clases y paquetes que forman parte del núcleo de la aplicación.
3. `src/main/resources`: Este directorio contiene los recursos adicionales que no son código Java, como archivos de configuración, archivos de propiedades, archivos de texto, etc. Este directorio es opcional y solo se incluye si el proyecto requiere estos recursos.
4. `src/test/java`: Este directorio alberga el código fuente de las pruebas del proyecto, generalmente escritas en Java utilizando frameworks como JUnit o TestNG.
5. `src/test/resources`: Similar al directorio `src/main/resources`, este directorio contiene los recursos adicionales necesarios para las pruebas, como archivos de configuración de pruebas.
6. `pom.xml`: Este es el archivo central del proyecto Maven. Es un archivo XML que contiene la configuración del proyecto, incluyendo información sobre las dependencias, plugins, configuraciones de compilación, entre otros. Es aquí donde se define la estructura del proyecto y cómo se debe construir.
7. `target`: Este directorio es generado automáticamente por Maven durante la construcción del proyecto. Contiene los archivos generados como resultado del proceso de compilación, como los archivos `.class`, los archivos `.jar`, los archivos `.war`, entre otros.

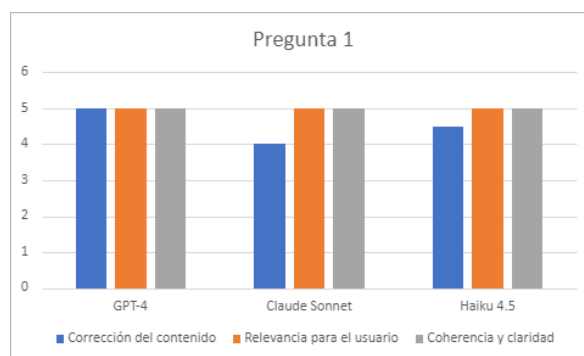


Figura 7.2: Resultados de la evaluación con el método "LLM as judges" para la primera pregunta

A los jueces les ha gustado esta respuesta coinciden en que es directa, aborda la pregunta del usuario sin desviarse y el contenido es correcto.

- Respuesta a la pregunta 2:

1. Identifica las dependencias externas: Primero, debes identificar las dependencias externas de tu SUT. Estas son las clases o servicios que tu SUT utiliza y que podrían ser difíciles de controlar durante las pruebas.
2. Crea un stub para cada dependencia externa: Un stub es una clase que simula el comportamiento de una dependencia externa. Debes crear una clase stub que extienda la clase de la dependencia real y sobrescribir los métodos que necesitas controlar. Por ejemplo, si tu SUT depende de una clase `Buscador`, podrías crear un `BuscadorStub` que extienda `Buscador` y sobrescribir los métodos que necesitas controlar.

```
java public class
BuscadorStub extends Buscador { private int result; public void
setResult(int salida) { this.result = salida; } @Override public int
buscarAlgo() { return result; } }
```
3. Modifica tu SUT para inyectar los stubs: Necesitas modificar tu SUT para que pueda aceptar una instancia de la dependencia real o de su stub. Esto generalmente se hace a través de la inyección de dependencias. Puedes hacer esto pasando la instancia de la dependencia al constructor de tu SUT o a través de un método setter.

```
java public class
GestorPedidos { private Buscador buscador; public GestorPedidos(
Buscador buscador) { this.buscador = buscador; } public void
procesarPedidos() { int resultado = buscador.buscarAlgo(); // Procesa
el resultado } }
```
4. Implementa el driver: El driver es la clase que se encargará de configurar los stubs y ejecutar el SUT con los datos de entrada diseñados. En tu caso, el driver podría ser una clase de prueba como `GestorPedidosTest`.

```
java public class GestorPedidosTest { @Test public
void testProcesarPedidos() { BuscadorStub buscadorStub = new
BuscadorStub(); buscadorStub.setResult(10); // Configura el resultado
del stub GestorPedidos gestorPedidos = new GestorPedidos(buscadorStub);
gestorPedidos.procesarPedidos(); // Verifica el estado resultante //
Por ejemplo, puedes verificar si el SUT procesó correctamente el
resultado } }
```
5. Verifica el estado resultante: Finalmente, debes verificar el estado resultante de la invocación de tu SUT. Esto puede implicar comprobar si el SUT procesó correctamente los datos de entrada proporcionados por el stub. Siguiendo estos pasos, podrás implementar un driver que use stubs para controlar las dependencias externas de tu SUT durante las pruebas.

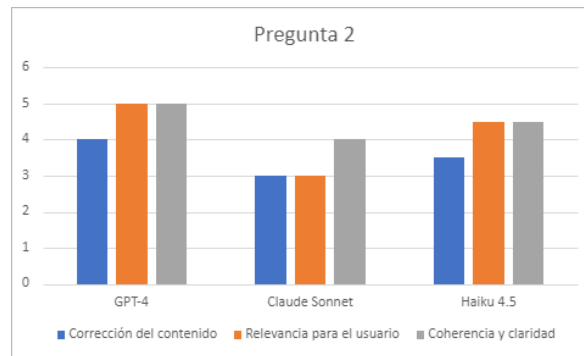


Figura 7.3: Resultados de la evaluación con el método "LLM as judges" para la segunda pregunta

Para esta pregunta los jueces coinciden en que el contenido es correcto de forma general, sin embargo, es imprecisa y tiene errores menores. El más estricto ha sido Sonet, que ha expuesto lo siguiente:

- Hay un error conceptual importante que afecta al núcleo de la respuesta: el tutor confunde parcialmente los roles de driver y stub. En la teoría de pruebas por integración incremental:
 - * El stub sustituye a un módulo subordinado (llamado por el SUT). Esto lo explica correctamente
 - * El driver sustituye al módulo superior que invoca al SUT. El tutor lo describe como "la clase de prueba JUnit que configura los stubs y ejecuta el SUT", lo cual es en realidad el rol del test harness o clase de prueba, no estrictamente el driver en sentido clásico.
- Además, el stub se implementa mediante herencia (extends Buscador), lo cual es funcionalmente válido pero menos deseable que implementar una interfaz, ya que acopla el stub a la implementación concreta. Esta no es un error grave pero sí una omisión de buenas prácticas. El resto del contenido (inyección de dependencias, configuración del stub, verificación) es correcto.

Los otros jueces, aunque más laxos han coincidido en que la respuesta es mejorable.

• Respuesta la pregunta 3:

Un test parametrizado es una técnica de pruebas en la que un único test se ejecuta múltiples veces con diferentes conjuntos de datos. Esto permite probar varias entradas y salidas sin tener que escribir un test separado para cada caso. Es especialmente útil cuando el código de varios tests es idéntico, salvo por los valores concretos de los casos de prueba. Aquí tienes un ejemplo de cómo implementar un test parametrizado usando JUnit 5 en Java:

```
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.MethodSource;
import java.util.stream.Stream;
```

```
import static org.junit.jupiter.api.Assertions.assertEquals;

public class EjemploTestParametrizado {

    @ParameterizedTest
    @MethodSource("casosDePrueba")
    void testSuma(int a, int b, int resultadoEsperado) {
        int resultado = a + b;
        assertEquals(resultadoEsperado, resultado);
    }

    static Stream<Arguments> casosDePrueba() {
        return Stream.of(
            Arguments.of(1, 2, 3),
            Arguments.of(5, 7, 12),
            Arguments.of(-1, -1, -2),
            Arguments.of(0, 0, 0)
        );
    }
}
```

En este ejemplo, el método `testSuma` es un test parametrizado que se ejecuta cuatro veces, una para cada conjunto de argumentos proporcionado por el método `casosDePrueba`. Cada ejecución del test utiliza diferentes valores para `a`, `b` y `resultadoEsperado`, permitiendo probar la función de suma con varios casos de prueba diferentes.

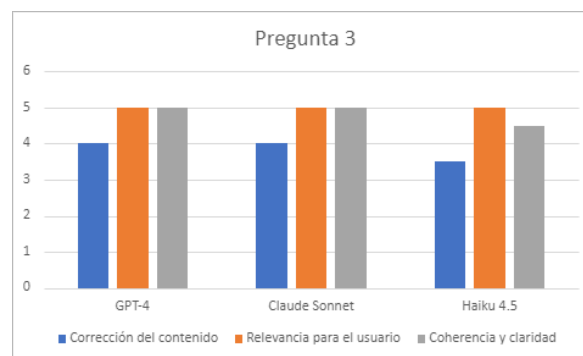


Figura 7.4: Resultados de la evaluación con el método "LLM as judges" para la tercera pregunta

Los resultados obtenidos muestran que los jueces han valorado mejor la respuesta a la pregunta 1, que es teórica, mientras que las preguntas con partes de código han obtenido peores resultados, de igual forma que en los resultados de BERTScore, lo que podría indicar que las respuestas en relación a la programación tienen rango de mejora.

GPT-4 Ha sido el que ha evaluado de forma más positiva las respuestas, mientras que Sonnet-4.5 ha sido más estricto.

7.2 Encuestas a usuarios

Para comprobar la utilidad de la aplicación en un entorno real, se ha compartido la aplicación con 8 alumnos de ingeniería informática. El objetivo del test es comprobar la experiencia de aprendizaje de alumnos reales con el tutor y su satisfacción con el sistema. La interfaz no se ha tenido en cuenta para el cuestionario pues se usa Chainlit. Los resultados obtenidos son los mostrados en la figuras 7.5 y 7.6.

La mitad de los usuarios encuestados han valorado como "mejor" o "mucho mejor" la aplicación en comparación a otras formas de estudio por su capacidad para practicar con preguntas. El resto la considera similar a la experiencia que puede ofrecer ChatGPT o Claude, aplicaciones que ya usan. En general el nivel de satisfacción entre los usuarios es alto, en especial a la hora de preguntar dudas al tutor y adquirir conocimiento nuevo. Las preguntas que plantea el tutor han gustado entre los usuarios, aunque algunos no están seguros de que el tutor pueda adaptarse realmente a su nivel.

Por último, se ha pedido a los usuarios que den su opinión.

Algunos se han quejado de asperezas presentes en el tutor, como que no guarde un historial de conversaciones tras cerrar la aplicación, o los tiempos de espera en algunos mensajes. La mayoría coinciden en que podría ser útil pero que necesitaría más tiempo para desarrollarse por completo para poder ser explotada a nivel comercial y usada normalmente por el alumnado. Varias de las quejas tienen fácil solución y están alejadas de los objetivos del proyecto.

El resto han valorado positivamente el tutor en su estado actual y creen que podría ser útil para reforzar conocimientos aprendidos.

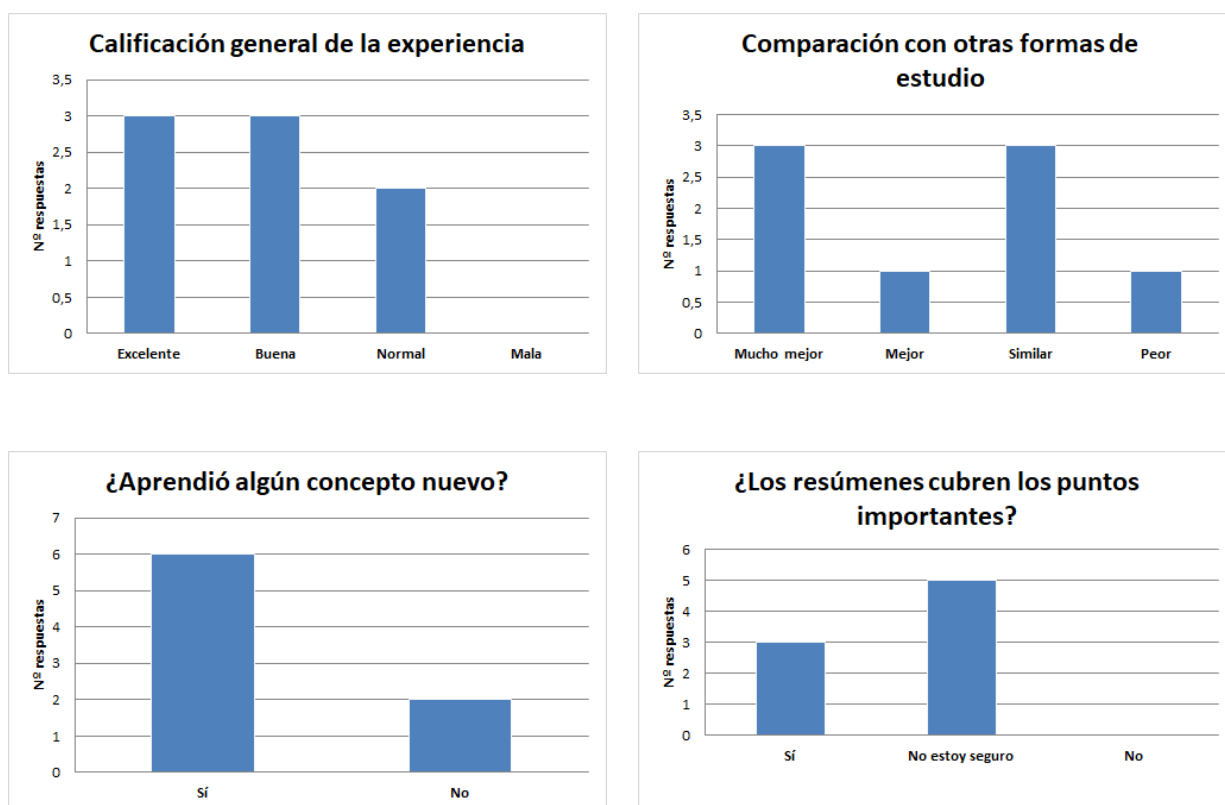


Figura 7.5: Resultados de las encuestas a usuarios (1).

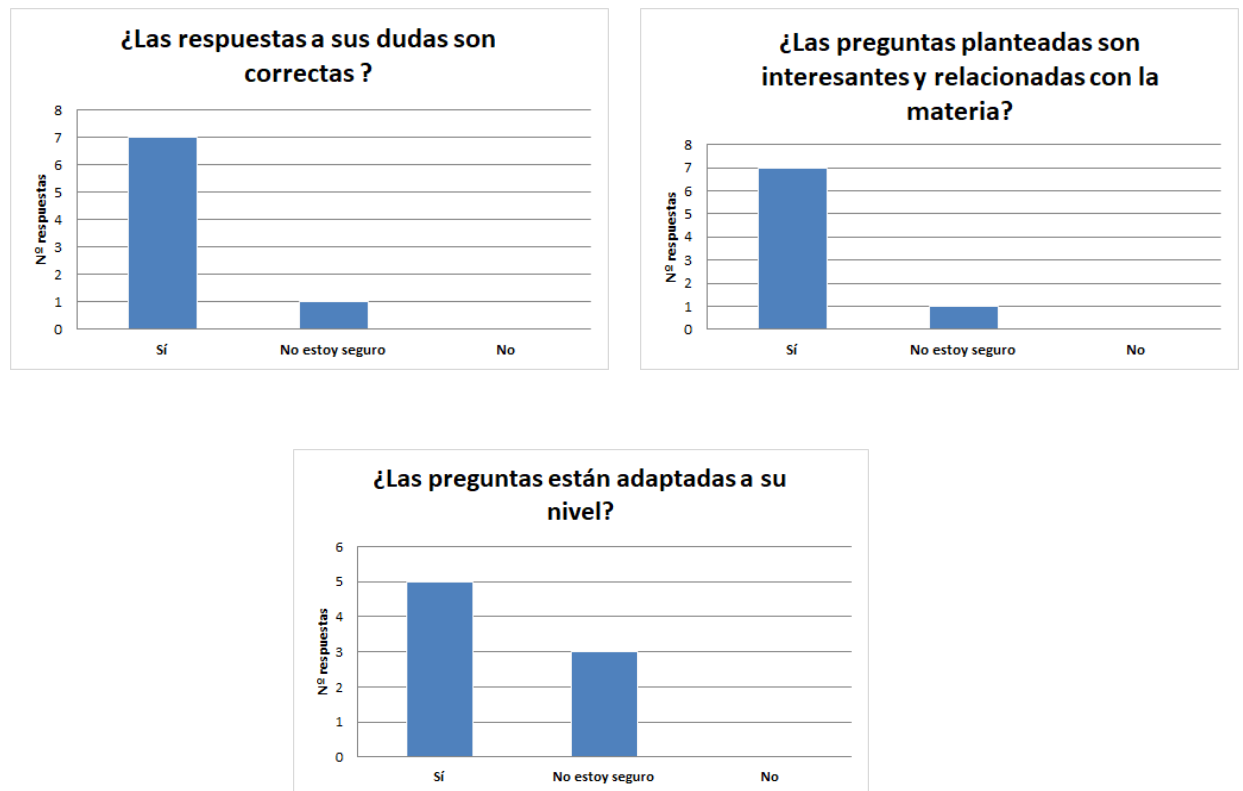


Figura 7.6: Resultados de las encuestas a usuarios (2).

8 Conclusiones

Tras finalizar el proyecto, se cuenta con un prototipo funcional, que actúa de tutor inteligente virtual para ayudar a los alumnos de informática en el aprendizaje, pudiendo resolver dudas, generar preguntas, evaluar sus respuestas y resumir contenidos de los materiales docentes. Cuenta con RAG para la recuperación de información, y es capaz de adaptarse al nivel del usuario para generar las preguntas y evaluar su conocimiento con sistemas reales como IRT y CBM.

Además, desde un ámbito personal estoy satisfecho con el trabajo realizado. Al no tener conocimientos previos sobre LLMs, ha supuesto una oportunidad para entender y aprender su funcionamiento interno y como aplicarlo para crear aplicaciones interesantes y complejas que puedan asistir a la gente en diferentes aspectos de la vida, en este caso, dentro de un marco estudiantil y universitario. Valoro haber aprendido sobre esta materia, especialmente teniendo en cuenta la importancia presente y futura que tendrán estos sistemas en nuestras vidas.

8.1 Problemas enfrentados

El principal problema ha sido la limitación de tener que trabajar con modelos LLM de menor nivel que los profesionales de pago que se ofrecen en el mercado, esto limita los resultados y las capacidades del tutor, sin embargo no es posible para un proyecto de este tamaño lanzar esa clase de modelos que necesitan de infraestructuras de servidores potentes. En especial, el modelo Qwen3-0.6B, el usado durante el desarrollo para lanzar el LLM en local, es demasiado pequeño, y ha complicado el desarrollo debido a la simplicidad de las respuestas que genera.

8.2 Posibles mejoras en futuras versiones

Para finalizar, creo conveniente aclarar que el prototipo es acorde a los objetivos planteados, sin embargo, hay espacio para implementar mejoras en el proyecto, en el caso de continuar el desarrollo.

- **Mejoras pedagógicas:** El programa es capaz de adaptarse al nivel del usuario con las preguntas, aumentando o disminuyendo la dificultad y siguiendo los niveles de Bloom. Esto se considera suficiente para los objetivos y escala del proyecto en relación a las restricciones de tiempo para la entrega. Sin embargo, en comparación con algunos ITS profesionales y comerciales publicados, puede ser simple de cara a un supuesto lanzamiento oficial del proyecto. Sería interesante añadir diferentes estrategias pedagógicas al tutor, para que cuando un usuario pregunte una duda, el tutor actúe de forma socrática, ayudando al usuario a completar el conocimiento en lugar de explicar directamente la solución. Otro aspecto que no trata este TFG es la ludificación (o "gamificación")

como forma de mejorar el rendimiento del estudiante. Huang y cols. (2025) en su investigación determinaron que los ITS con procesos ludificados mejoraban los resultados medidos de los estudiantes respecto aquellos que no los tenían.

- **Mejoras en relación a las preguntas:** El sistema de evaluación de preguntas se basa en la dificultad que el propio LLM estime para las preguntas que genera. Esto presenta un problema, pues se confía que este valor es correcto, pero bien podría no estar del todo ajustado a la dificultad real de la pregunta. Para solventarlo, algunos sistemas proponen la misma pregunta a muchos usuarios con distintos niveles de conocimiento medido, observan si los alumnos son capaces de responder bien y van ajustando la dificultad. Sin embargo, es difícil de replicar en este momento del proyecto sin una base grande usuarios interactuando con la aplicación.
 - **Optimización:** Como se ha visto al analizar las métricas, los tiempos de espera de respuestas por parte de la IA son elevados en algunas de las operaciones. Este es un problema común en muchos sistemas basados en LLM en la actualidad, debido a la enorme complejidad de computo de estos sistemas, sin embargo, de cara a mejorar la experiencia de los usuarios, sería conveniente estudiar qué partes del código son mejorables para reducirlos.
 - **Mejoras gráficas:** Chainlit es útil de cara a presentar el prototipo sin la necesidad de centrarse en el frontend sin embargo, en el futuro debería dejarse de lado para realizar una interfaz profesional y desarrollada para la aplicación. También, incluyendo manejo de roles de usuarios, las funciones como generar embeddings podrían presentarse de forma gráfica a los usuarios administradores de la aplicación, en lugar de por consola como se encuentra actualmente.
-

9 Anexo

9.1 Prompt usado en LLM As Judges

```
[System]
Eres un evaluador experto de sistemas tutores inteligentes (ITS).
Tu tarea es evaluar la respuesta de un tutor IA a la pregunta
de un estudiante.

DOMINIO: {Asignatura}
PASO 1 -RESPONDE TÚ PRIMERO (no leas aún la respuesta del tutor)
Antes de evaluar, responde brevemente la pregunta del estudiante
con tu propio conocimiento. Esto servirá como referencia interna.

PREGUNTA DEL ESTUDIANTE:
{mensaje_estudiante}

TU RESPUESTA DE REFERENCIA: [escribela aquí antes de continuar]

---

PASO 2 -EVALÚA LA RESPUESTA DEL TUTOR
Ahora lee la respuesta del tutor y evalúa comparándola con
tu referencia interna.

RESPUESTA DEL TUTOR:
{respuesta_tutor}

---

CRITERIOS DE EVALUACIÓN (uno por uno, sin dejarte influir
por la longitud de la respuesta):

CRITERIO 1 -CORRECCIÓN DEL CONTENIDO
¿Las afirmaciones son factualmente correctas?
¿Hay errores, imprecisiones u omisiones importantes?
RAZONAMIENTO: [tu análisis]
PUNTUACIÓN: [[1-5]]

CRITERIO 2 -RELEVANCIA PARA EL USUARIO
¿La respuesta aborda exactamente lo que el estudiante preguntó?
¿Se desvía o ignora parte de la pregunta?
RAZONAMIENTO: [tu análisis]
PUNTUACIÓN: [[1-5]]
```

CRITERIO 3 -COHERENCIA Y CLARIDAD

¿La explicación tiene estructura lógica y es comprensible para un estudiante?

RAZONAMIENTO: [tu análisis]

PUNTUACIÓN: [[1-5]]

9.2 Ejemplos de conversaciones

Ejemplo de uso 1: Un usuario pide un resumen de un tema

```
.....
Bienvenido! Soy un tutor virtual inteligente.
.....
Hola
.....
¡Hola! ¿En qué puedo ayudarte hoy?
.....
Hazme un resumen del tema 2 de ppss
.....
### Resumen del Tema 2: Diseño de Pruebas (Caja Blanca)

#### 1. Introducción
- **Objetivo**: Obtener una tabla de casos de prueba que detecte el máximo número de defectos con el mínimo número de casos de prueba.
- **Concepto de Caja Blanca**: Diseño de casos de prueba basado en el código y la implementación del programa.

#### 2. Diseño de Casos de Prueba
- **Tabla de Casos de Prueba**: Cada fila contiene datos de entrada y resultado esperado.
- **Objetivo**: Determinar valores de entrada y resultados esperados a partir de la implementación y especificación del programa.

#### 3. Métodos Estructurales de Diseño de Casos de Prueba
- **Control Flow Testing**: Método que utiliza el flujo de control del programa para diseñar casos de prueba.
- **Paso 1**: Análisis de código y construcción del CFG (Grafo de Flujo de Control).
- **Paso 2**: Selección de caminos en el CFG.
- **Paso 3**: Obtención de casos de prueba que ejerciten dichos caminos.

#### 4. Conceptos Clave
- **CFG (Grafo de Flujo de Control)**: Representación gráfica de los comportamientos de una unidad de programa.
- **Camino Independiente**: Camino en el CFG que difiere de otros en al menos un conjunto de sentencias y/o una nueva condición.
- **Complejidad Ciclomática (CC)**: Métrica que proporciona una medida de la complejidad lógica del programa.
- **Método del Camino Básico**: Método que permite ejercitar cada camino independiente en el programa.
```

```
#### 5. Cálculo de CC
- **Fórmula**: CC = número de arcos - número de nodos + 2.
- **Significado**: Indica el número máximo de caminos independientes en el CFG.

#### 6. Selección de Caminos
- **Criterios**: Ejecutar todas las sentencias al menos una vez, todas las
  condiciones al menos una vez.
- **Objetivo**: Ejercitar el máximo número de comportamientos del programa.

#### 7. Ejemplo de Aplicación: Método del Camino Básico
- **Paso 1**: Construir CFG.
- **Paso 2**: Calcular CC.
- **Paso 3**: Obtener caminos independientes.
- **Paso 4**: Determinar datos de entrada y resultados esperados.

#### 8. Ejercicios Propuestos
- **Cálculo de CC**: Aplicar la fórmula a diferentes códigos Java.
- **Diseño de Casos de Prueba**: Utilizar el método del camino básico para diseñar
  casos de prueba.

#### 9. Referencias Bibliográficas
- **Libros**: "Pragmatic Software Testing" de Rex Black, "Software Testing and
  Quality Assurance" de Kshirasagar Naik & Priyadarshi Tripathy, "A 'Practitioners
  Guide to Software Test Design" de Lee Copeland.

### Puntos Esenciales para Comprender el Tema
- **Entender CFG y su construcción**.
- **Calcular CC y su importancia en el diseño de pruebas**.
- **Aplicar el método del camino básico para diseñar casos de prueba**.
- **Ejercitar todos los comportamientos del programa**.

Este resumen proporciona una visión clara y concisa de los conceptos y métodos clave
para el diseño de pruebas de caja blanca, enfocándose en la construcción de
casos de prueba efectivos y eficientes.
.....
```

Ejemplo de uso 2: El modelo resuelve una duda al alumno:

```
.....
Explícame que son las pruebas de caja negra
.....
Las pruebas de caja negra, también conocidas como pruebas de diseño de caja negra,
son un método de prueba de software que se enfoca en el comportamiento del
programa desde la perspectiva del usuario final, es decir, desde la entrada
hasta la salida sin considerar la implementación interna del programa. Este
método se basa en las especificaciones del programa y se utiliza para verificar
que el programa funcione correctamente para todos los posibles valores de
entrada.

En el contexto de las pruebas de caja negra, se considera al programa como una "caja
negra" porque el desarrollador no tiene acceso a su implementación interna;
solo se conoce su entrada y salida. El objetivo es diseñar casos de prueba que
```

cubran todos los posibles comportamientos del programa según sus especificaciones.

Existen varios métodos para diseñar casos de prueba de caja negra, como:

1. ****Método de particiones equivalentes****: Divide los datos de entrada en clases de equivalencia y luego elige un representante de cada clase para los casos de prueba.
2. ****Método de análisis de valores límite****: Se centra en los valores límites de los rangos de entrada y salida, ya que estos son puntos donde el comportamiento del programa puede cambiar.
3. ****Método de tablas de decisión****: Utiliza las condiciones y decisiones dentro de un programa para diseñar casos de prueba que cubran todas las combinaciones posibles.
4. ****Método de diagramas de transición de estados****: Se utiliza para programas que tienen estados y transiciones, asegurando que todas las transiciones y estados se prueben.
5. ****Método de pruebas basado en casos de uso****: Se enfoca en los casos de uso del sistema para asegurar que todas las funcionalidades se prueben.
6. ****Método de pruebas basado en requerimientos****: Se basa en los requerimientos del sistema para diseñar casos de prueba que cubran todas las funcionalidades especificadas.

Es importante destacar que los casos de prueba obtenidos mediante este método son independientes de la implementación del programa y pueden ser diseñados antes de la implementación. Esto permite una mayor eficiencia en el proceso de pruebas y asegura que se cubran todos los posibles comportamientos del programa.

.....
Y si tuviese acceso a ver el código para diseñar las pruebas estáticas, que otro método puedo usar

.....
Si tienes acceso al código para diseñar pruebas estáticas, puedes utilizar el método de Diseño de Pruebas de Caja Blanca basado en el flujo de control. Este método, también conocido como Método del Camino Básico o Método de Cobertura de Caminos, es muy útil para asegurar que se prueben todos los caminos posibles en el programa.

Pasos para Diseñar Pruebas de Caja Blanca usando el Método del Camino Básico:

1. ****Representación del Código en un Grafo de Flujo de Control (CFG):****
 - ****CFG****: Es una representación gráfica de los comportamientos de una unidad de programa. Usaremos un grafo dirigido donde cada nodo representa una instrucción o una decisión, y las aristas representan las transiciones entre ellas.
2. ****Selección de Caminos:****

```
- **Camino Básico**: Selecciona un camino básico que cubra todas las
    ramificaciones del código. Un camino básico es un conjunto de instrucciones
    que se ejecutan en un orden determinado y que cubre todas las ramificaciones
    del código.

3. **Generación de Casos de Prueba**
    - **Cobertura de Caminos**: Obtén un conjunto de casos de prueba que ejerciten
      cada camino básico identificado en el CFG. Esto asegura que se prueben todas
      las ramificaciones posibles del código.

### Ejemplo:

Supongamos que tienes el siguiente código en Java:

```java
public int divide(int a, int b) {
 if (b == 0) {
 return -1; // Error: Division by zero
 } else {
 return a / b;
 }
}
```

### Paso 1: Representación en un CFG

El CFG para este código sería:



```

1. Inicio
2. if (b == 0) goto 4
3. else goto 6
4. return -1
5. goto 7
6. return a / b
7. Fin

```



### Paso 2: Selección de Caminos



```

- **Camino 1**: `Inicio -> 2 -> 4`
- **Camino 2**: `Inicio -> 3 -> 6`

```



### Paso 3: Generación de Casos de Prueba



```

- **Camino 1**: `b == 0` (prueba para división por cero)
- **Camino 2**: `b != 0` (prueba para una división normal)

```



### Casos de Prueba Generados:



```

1. **Prueba 1**: `b = 0`

```


```

```

- Entrada: `a = 10`, `b = 0`
- Esperado: `return -1`

2. Prueba 2: `b != 0`
- Entrada: `a = 10`, `b = 1`
- Esperado: `return 10`

### Observaciones:

- Eficiencia: Este método es eficiente porque genera un conjunto mínimo de casos de prueba que cubren todos los caminos posibles.
- Complejidad: Aunque es eficiente, puede ser costoso para programas complejos con muchos caminos.

### Conclusión:

Usar el Método del Camino Básico te permite diseñar pruebas que cubren todos los caminos posibles en el código, lo que es crucial para detectar errores relacionados con las ramificaciones y las condiciones de control. Sin embargo, recuerda que este método no puede detectar todos los defectos posibles, ya que hay otros factores que pueden afectar la cobertura de pruebas.

Espero que esta explicación te ayude a diseñar tus pruebas de caja blanca de manera efectiva. Si tienes más preguntas, no dudes en preguntar.
.....

```

Ejemplo de uso 3: El tutor realiza un cuestionario al alumno.

```

.....
Bienvenido! Soy un tutor virtual inteligente.
.....
Hazme dos preguntas del tema 1
.....
Pregunta: ¿Qué es un build script en el proceso de construcción de un sistema?
A: Un conjunto de instrucciones para la compilación del código fuente
B: Un conjunto de instrucciones para la limpieza del código fuente
C: Un conjunto de instrucciones para la documentación del código fuente
D: Un conjunto de instrucciones para la optimización del código fuente
Escriba la letra de la opción que considere correcta:
a
¿ Cómo de seguro está de su respuesta? No está seguro (1), seguro a medias (2),
    bastante seguro (3)
3
¡Respuesta correcta!

¿Cuáles son los beneficios de la automatización en el diseño y ejecución de pruebas de software?
Facilitar el testeo del código al reducir costes y tiempo de trabajo para probarlo,
    hanciendo que se pueda probar el software de forma sencilla
FALSO: La respuesta contiene errores gramaticales y es demasiado incompleta y vaga

```

para explicar los beneficios de la automatización en pruebas de software. Los beneficios de la automatización en el diseño y ejecución de pruebas de software incluyen la reducción de costos y tiempo, la mejora de la precisión y consistencia de las pruebas, la capacidad de ejecutar pruebas repetitivas de manera más eficiente, y la posibilidad de probar el software en condiciones que pueden ser difíciles o imposibles de replicar manualmente.

Preguntas acertadas:1/2

Bibliografía

- Armstrong, P. (2016). Bloom's taxonomy. *Center for Teaching Vanderbilt University*.
- Bloom, B. (1984). The 2 sigma problem: The search for methods of group instruction as effective as one-to-one tutoring. *Educational Researcher*.
- Bulut, O., Shin, J., Seyma N. Yildirim-Erbasli, G. G., y Pardos, Z. A. (2023). An introduction to bayesian knowledge tracing with pybkt. *MDPI*.
- Carbonell, J., y Collins, A. M. (1970). *Mixed-initiative systems for training and decision-aid applications*. <https://eric.ed.gov/?id=ED049630>.
- Cataldi, Z., y Lage, F. J. (2009). Sistemas tutores inteligentes: Procedimientos, métodos, técnicas y herramientas para su creación. *virtual Educa Argentina*.
- Chainlit: Documentation*. (s.f.). <https://docs.chainlit.io/get-started/overview>.
- D'Mello, S., y Graesser, A. (2012). Autotutor and affective autotutor: Learning by talking with cognitively and emotionally intelligent computers that talk back. *ACM DIGITAL LIBRARY*.
- Elhaimeur, I., y Chrisochoides, N. (2026). From prototype to classroom: An intelligent tutoring system for quantum education. *arXiv*.
- Gardner-Medwin, T. (1990). *Certainty-based marking (cbm)*. <https://www.tmedwin.net/cbm/selftests/>.
- Guillen, A. G., Gómez, H. A., y Lopez, R. R. (2017). Modelo de un sistema tutor inteligente para el desarrollo del pensamiento computacional. *CIM Journal*.
- Huang, X., Xu, W., y Liu, R. (2025). Effects of intelligent tutoring systems on educational outcomes: Evidence from a comprehensive analysis. *Science Direct*.
- Liu, Z., Agrawal, P., Singhal, S., Madaan, V., Kumar, M., y Verma, P. K. (2025). Lpitudor: an llm based personalized intelligent tutoring system using rag and prompt engineering. *PMC*.
- Prompt engineering guide*. (s.f.). <https://www.promptingguide.ai/>.
- Teaching machine*. (s.f.). https://www.si.edu/object/pressey-teaching-machine:nmah_1367149.
- Two-parameter item response theory (2pl)*. (2026). <https://www.emergentmind.com/topics/two-parameter-item-response-theory-model>.

- Venegas, R. (2003). Análisis semántico latente: una panorámica de su desarrollo. *Revista Signos*.
- Vermeiren, H., Kruis, J., Bolsinova, M., van der Maas, H., y Hafman, A. (2025). Psychometrics of an elo-based large-scale online learning system. *Science Direct*.
- Vllm cli guide*. (s.f.). <https://docs.vllm.ai/en/stable/cli/>.
- Zhang, T., Kishore, V., Wu, F., Weinberger, K., y Artzi, Y. (2020). Bertscore: Evaluating text generation with bert. *arXiv*.
- Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., ... Stoica, I. (2023). Judging llm-as-a-judge with mt-bench and chatbot arena. *arXiv*.
-